Implementing and Improving YOLOv11 to detect wildfires using UAV imagery

Kevin Joy

BSc in Computer Science with AI
The University of Bath
2025

Implementing and Improving YOLOv11 to detect wildfire using UAV imagery

Submitted by: Kevin Joy

Copyright

Declaration

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in Computer Science with AI in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

Abstract

<The abstract should appear here. An abstract is a short paragraph describing the aims of the project, what was achieved and what contributions it has made.>

Contents

Abstract

Wildfires have become increasingly frequent and severe due to climate change, posing urgent risks to ecosystems and human settlements. Traditional detection systems, such as satellites and static sensors, suffer from low resolution, long revisit times, and poor scalability. This study explores the application of UAV-based vision systems for real-time wildfire detection using deep learning, with a focus on improving the YOLOv11 architecture for enhanced performance. A custom, balanced dataset derived from the D-Fire corpus was used to train and test several YOLO-based models, including baseline YOLOv5, YOLOv8, YOLOv10, and YOLOv11 variants, as well as attention-augmented configurations. Among them, a YOLOv11 model integrated with a Convolutional Block Attention Module (CBAM)—using a minimal-efficiency strategy—delivered the best overall performance. It achieved the highest F1 score (0.704) and mAP@50 (0.703) while maintaining real-time inference speed (56.98 FPS), outperforming all other variants. These results underscore the value of strategically placed attention mechanisms, which enhanced spatial and channel awareness without significant computational overhead. Alternative backbones such as ConvNeXt and EfficientNet were also tested, but did not surpass the default YOLOv11 backbone, suggesting that architectural coherence plays a crucial role in model efficiency. Transfer learning strategies involving frozen layers provided negligible benefit and were outperformed by full fine-tuning. Overall, the optimized CBAM-YOLOv11 model presents a robust, lightweight, and deployable solution for early wildfire detection from aerial platforms, bridging the gap between detection accuracy and real-time field application. This work demonstrates that targeted architectural enhancements can significantly boost object detection performance for critical real-world scenarios such as wildfire monitoring.

Acknowledgements

I would like to thank my supervisor Dr. Michael Ying Yang and Professor Wenbin Li for helping me navigate my way through this project.

Chapter 1

Introduction

Wildfires are escalating in frequency and intensity worldwide, mainly due to climate change. NASA's analysis conclude that "extreme wildfire activity has more than doubled worldwide" over the past two decades(NASA, n.d.). While global carbon emissions from forest fires rose about 60% between 2001 and 2023. Regions like the western United States have seen historic surges and wildfires have increased around 400% over recent decades. These statistics underline urgent need for advanced fire detection and prevention. Early identification of small fire or smoke plumes can dramatically enhance response times, consequently reducing loss of life, property and ecosystems. In 2018, Camp Fire in California burnt over 620 square kilometres, resulting in 85 deaths, with $16.5B in damages. This illustrates how even a few hours of early warning marks the difference between a contained incident and a categorical catastrophe. Hence, the motivation for this work is clear, with wildfires frequency on the rise, developing faster, more precise detection methods is a critical step toward mitigating their devastating effects.

Traditional detection methods rely on a mix of, of ground observations, remote sensing and fixed sensor networks, each with significant drawbacks. Satellite platforms (e.g. MODIS, VIIRS) provide broad coverage, but their coarse spatial resolution and infrequent revisits, severely limits early detection. In boreal forests, modelling shows 1 km satellite pixels (such as MODIS or SLSTR) would fail to would fail to detect roughly 44–70% of smouldering fires under the canopy (Johnston, et al., n.d.). Satellite data is typically aggregated to one pixel per fire (often ≥375 m), causing nascent flames or smoke to be averaged out and go unnoticed. Additionally long revisit intervals mean that a small fire can have already uncontrollably grown.

In practice, satellite-based alerts lack real-time responsiveness: their coarse resolution, limited coverage, and delay in data downlink render them unsuitable for continuous, real-time monitoring (Akhloufi, et al., 2021). ground-based networks (fixed cameras, heat/smoke detectors) have limited range, high deployment costs, and are vulnerable to damage upon fire exposure, with their static nature also restricting scalability across vast forests. This underscores limitations of conventional methods and highlights the need for high-resolution, rapid monitoring detection modalities, particularly in remote regions where early response is especially critical.

Unmanned aerial vehicles (UAVs), or drones offer a compelling solution to satellites and fixed sensors limitations. Rapidly deployable over vulnerable areas, UAV's provide real-time, high-resolution imagery. The manoeuvrability of drones "can navigate challenging terrain" delivering dynamic, high-frequency data, especially valuable in rugged or inaccessible regions. UAVs are lightweight, affordable and cost-effective, offering vast coverage, meaning examples like RGB cameras, infrared/thermal imagers and LiDAR can detect fires at various stages. Thermal cameras on drones can spot heat signatures through smoke or at night, while Lidar helps mapping terrain and fuel loads to predict fire spread. These characterises make UAVs ideal, making it the technology of choice for future fire monitoring. UAVs combine the best of satellite and ground methods, with high spatial detail and rapid revisit, exactly where and when it is needed.

Fire detection technologies have evolved following trends in sensing and machine learning. Early methods, manned lookout towers, fixed infrared scanners, computer vision applied to static camera feeds relied on human oversight. The rise of machine and deep learning has seen a revolution. Convolutional neural network (CNN) models transformed fire and smoke recognition, leveraging its complex visual pattern learning abilities. Notably, real-time object detection frameworks like YOLO ("You Only Look Once"), a single-shot detection model (first proposed in 2016) predicts bounding boxes and class probabilities in one forward pass, offering faster speed than older two-stage detectors like Faster R-CNN. YOLO rapidly evolved, with subsequent versions (v2 onwards) improving accuracy and multi-scale detection. Studies have applied YOLO to UAV-based fire surveillance, with (Demir et 2019) al running YOLOv8 and YOLOv5 onboard a Jetson nano, achieving 89-96% accuracy in detecting forest fires. These advances reflect deep learning, especially YOLO, has become a state-of-the-art approach for automated wildfire detection, providing speed and robustness required for real-time applications.

Among latest object detection algorithms, YOLOv11, launched by Ultralytics in 2024, emerges as a particularly promising choice. Key enhancements include an optimized backbone and neck for enhanced feature extraction, and a redesigned detection head optimized for minimal latency. YOLOv11 uses an anchor-free, decoupled head, separately handling classification and localization (Ultralytics, 2025) with refinements to accelerate inference. These innovations mean maintaining real-time performance on both GPUs and smaller edge devices, squeezing out detection precision. It combines the best aspects of the YOLO family with new modules for efficiency, making it well-suited for UAV deployment.

Chapter 2

Related Work

2.1 Introduction

Conventional sensor-based wildfire detection methods that utilize heat or smoke detectors often respond slowly and struggle in outdoor or wide-area settings (Ismail, et al., 2025). To address this, researchers have adopted vision-based wildfire detection systems to automatically recognise fire or smoke in real time.
This section of the related work outlines the traditional image-processing techniques based on hand-crafted rules, classical machine learning methods using engineered features and various deep learning approaches using neural networks. With the focus being on real-time systems for wildfire detection, highlighting their performance and limitations in terms of accuracy, false positives and negatives, scalability and computational cost.

Traditional Computer Vision Techniques

Early vision-based fire detection models relied on manually crafted rules to identify visual fire cues like flame colour, motion, and shape. These approaches implement thresholding colour channels or using simple colour models in the RGB, HSV or YCbCr spaces to isolate the characteristic orange-red hues of a flame (Yoon-Ho Kim, 2014). While effective in controlled environments, pure colour-based methods often proved not viable in varying illumination and backgrounds. To enhance reliability, researchers further incorporated dynamic cues. These cues include the erratic flickering and spreading nature of flames, as well as the drifting and the semi-transparent plumes characteristic of smoke.
Traditional algorithms often combined multiple features that include colours, textures, shapes, and motion to better distinguish fire from confounding phenomena. (B. Ugur T ˇ oreyin, 2005), used wavelet transform analysis to capture the characteristic flicker frequency of flames and the motion patterns of smoke in video.

These traditional techniques typically ran efficiently on modest hardware and required no training data. However, their reliability was limited. They relied on fixed thresholds and simplistic assumptions, making them sensitive to environmental changes like sunlight, reflections or camera motions that could trigger false positives as well as variable appearances of fire and smoke.

Classical Machine Learning Methods

To improve upon the traditional methods, classical machine learning techniques were also introduced. These models used hand-crafted visual features as input to statistical models. Instead of rigid thresholds, these methods learned decision boundaries between “fire” and “no fire” using labelled data. Common features included colour histograms, texture descriptors, shape features and even temporal motion characteristics. These features were then made inputs for classifiers such as Support Vector Machines (SVMs) or Random Forests (RF). (Yusuf Hakan Habiboğlu, 2011), divided video frames into Spatio-temporal blocks and extracted covariance-based descriptors to capture dynamic textures. They then followed to train an SVM to distinguish fire vs. non-fire regions. Likewise, (Dimitropoulos, et al., 2015) combined colour, flicker, and region-growth features over time and fed them into an SVM to classify candidate regions as flame or not.

Research has also demonstrated the value of ensemble learning techniques in forest fire susceptibility modelling. (Ljubomir Gigović, 2019) proposed a hybrid approach combining SVMs and RF classifiers to assess fire-prone areas within Tara National Park, Serbia. This method yielded an AUC value of 0.848, indicating a notable improvement in performance. These findings outline the potential of integrated machine learning models to enhance the reliability of fire risk assessments, particularly when compared to traditional heuristic approaches.

Despite these advances, classical ML approaches still had notable constraints. They remained dependent on the quality of the chosen features. If important visual cues were not captured, the classifier could fail. Moreover, many ML-based detectors were tested on relatively small or homogeneous datasets, so their generalization to broader wildfire conditions was uncertain. In practice, environmental variability could still lead to missed detections or false alarms when conditions fell outside the training set.

Other Deep Learning Approaches

The advancement of deep learning has brought a notable change in wildfire detection by enabling end-to-end feature learnings directly from imagery. Convolutional Neural Networks (CNNs) began to outperform classical methods by automatically extracting complex visual patterns indicative of fire or smoke. Unlike earlier systems, deep learning models can be trained on large image datasets to recognize subtle fire cues without manual feature engineering.

Early CNN-based systems treated fire detection as an image classification task. For instance, (Frizzi, et al., 2016) trained a 6-layer CNN to classify images as fire, smoke or background, achieving higher accuracy than earlier multi-feature approaches. Similarly, (Tao, et al., 2016) and (Yin, et al., 2017) built deeper CNNs with 14 layers for smoke recognition, significantly improving early smoke detection accuracy over previous methods.

As datasets grew, object detection frameworks were applied to locate fires in scenes. (Qi-xing Zhang, 2018) addressed the scarcity of training data by inserting simulated smoke into forest images. Using this augmented dataset to train a Faster R-CNN model, they achieved accurate detection of smoke in wildland scenes. While

(Byoungjun Kim, 2019) combined a CNN-based region detector with a Long Short-Term Memory (LSTM) network to track fire occurrences across video frames, reducing false positives from the natural phenomena. Some approaches also introduce attention mechanisms to focus on smoke regions, (Gao Xu, 2019) proposed a saliency-driven deep network that highlights likely smoke areas before classifying the scene. Overall, deep learning methods dramatically improved detection performance have become the dominant approach in vision-based fire detection research.
However, these traditional deep learning models also introduced a set of new challenges. They require extensive labelled wildfire imagery for training which is difficult to obtain given the rarity of real wildfire events so techniques like data augmentation and synthetic data generation became important (Sousa, et al., 2019). Deep networks are also computationally intensive where a complex CNNs can struggle to run at full frame rate on standard CPUs or embedded devices. Moreover, even CNNs can yield false positives when confronted with visual inputs that differ from their training distribution (for example, clouds or sun glare resembling smoke).

Summary of Limitations and Ongoing Challenges:
Across all methods, several fundamental challenges remain in achieving reliable real-time wildfire detection. False positives remain a persistent problem, with environmental factors that can mimic the appearance of smoke or fire, causing frequent false positives if the algorithm isn't robust. Traditional and classical methods are especially sensitive to such interference, since their hand-crafted features cannot capture the full variability of real scenes. Deep learning models are more discriminative but can still misclassify unseen scenarios. Conversely, false negatives are still prevalent when fires are very small, distant or obscured hence reflecting a trade-off between sensitivity and specificity.
Scalability is also another concern. An algorithm that performs well on a single video feed may be challenging to deploy across dozens of camera feeds in a wide-area network. Classical methods were lightweight but less accurate, whereas deep CNNs offered high accuracy at the cost of greater computational load. Without optimization, complex models might not run in real time on edge devices, limiting practical applicability of these models.

2.2 Introduction to YOLO:
With the advancements in deep learning and the introduction to YOLO, real-time object detection has improved significantly with faster inference speeds, lower computation costs, and higher accuracy scores. In the following part of this section, we examine the previous state-of-the-art approaches focusing on YOLO-based architectures. The papers show how they have addressed the limitations and show how they compare to other CNN approaches and their enhancements specifically for UAV-based wildfire and smoke detection whilst the highlighting key innovations, performance metrics and limitations.

Evolution of CNN-Based Fire Detection:

This comparative study by (Pu Li, 2020) compares four advanced CNN based object detection models, these include: Faster-RCNN, R-FCN, SSD, and YOLOv3 for image-based fire detection. Using a dataset of 29,180 images containing 9,695 fire objects and 7,442 smoke objects, their evaluation demonstrated that YOLOv3 achieved the highest performance with 83.7% average precision while maintaining the fastest detection speed at 28 FPS. This balance between accuracy and speed made YOLOv3 particularly suitable for real-time applications on edge devices such as UAVs, where computational resources are constrained, and detection latency is critical.

This study clearly demonstrated the trade-off between computational demands and accuracy amongst the tested models. Two-stage image detectors models like Faster-RCNN provided higher accuracy but at significantly reduced speeds (3 FPS), rendering them impractical for time-sensitive UAV-based wildfire detection systems. Hence this research established the viability of YOLO architectures for UAV deployment and laid the groundwork for subsequent optimizations targeting the specific challenges of aerial wildfire detection.

YOLO-Based Model Developments and Enhancements:

Successive versions of the YOLO architecture have offered improvements in both accuracy and computational efficiency. (Leon Augusto Okida Gonçalves, 2024) conducted a comprehensive evaluation of a selection of YOLO-based models (YOLOv5, YOLOv5u, YOLOv7, YOLOv8) for detecting smoke and wildfires in both ground and aerial imagery. Testing on the D-Fire dataset (21,527 images) and WSDY dataset (737 smoke images) revealed that YOLOv7x achieved the best overall performance with 80.40% mAP while maintaining relatively fast detection speeds. For smoke-only detection, YOLOv8s achieved an impressive 98.10% mAP, demonstrating its potential for specialized UAV-based smoke monitoring applications. Hence showing that the later versions of the YOLO architecture perform better than the previous versions.

Several researchers have proposed architectural modifications to the base YOLO models to enhance the detection performance for the specific challenges posed by UAV-based wildfire monitoring. (Junhui Li, 2023) improved the YOLOv5s by integrating three key enhancements: a Coordinate Attention (CA) module to help focus on targets in different contexts, replacing the SPPF module with a Receptive Field Block (RFB) module to better capture global information of fires, and substituting the PANet in the neck structure with a Bi-directional Feature Pyramid Network (Bi-FPN) for improved feature fusion. Their optimized model achieved a 5.1% improvement in mAP@0.5 over the standard YOLOv5s, reaching 58.8% average precision.

Similarly, (Arafat Islam, 2023) proposed an improved YOLOv5 model focusing on small fire target identification across multiple environments. Their Fire-YOLOv5 achieved 90.5% mAP and 88% F1 score for small targets while processing 416×416

resolution images at 0.12 seconds per frame, enabling possible real-time detection capabilities suitable for UAV deployment.
This advancement is particularly significant for aerial fire detection, where targets often appear smaller due to the altitude of UAVs, but also shows growth for improvement.

Fine-Grained Feature Learning and Small Target Detection:

One of the most significant challenges with UAV-based wildfire detection is identifying small smoke and fire targets in the early stages of fires. (Long Sun, 2023) addressed this challenge with their ForestFireDetector, an improved YOLOv8-based architecture focusing on fine-grained feature learning. Their approach incorporated three key innovations: Space-to-Depth Convolution (SPD-Conv) modules to reduce information loss during down sampling, Ghost Shuffle Convolution (GSConv) for lightweight computation while maintaining feature extraction capability and an additional detection head specifically designed for small targets to leverage shallow feature information.
Their testing on 3,966 forest fire images demonstrated that the ForestFireDetector achieved 90.2% mAP, outperforming the baseline YOLOv8n by 3.3% while simultaneously reducing parameters from 3M to 2.7M. This improvement is particularly significant for UAV-based early-stage forest fire detection, where smoke instances are typically small and subtle against complex forest backgrounds. The reduction in model parameters also facilitates deployment on the resource-constrained computing platforms typically available on UAVs.
Following suite (Mukhriddin Mukhiddinov, 2022) focused on UAV-based early wildfire smoke detection with an optimized YOLOv5 model. They implemented several improvements, including K-mean++ technique for anchor box clustering, SPPF+ layer for small smoke region identification, and a BiFPN for faster multi-scale feature fusion. Their system achieved 73.6% average precision on a custom dataset of 6,000 wildfire images captured from UAVs. This implementation explicitly addresses the unique challenges of aerial perspectives, where smoke plumes exhibit different visual patterns compared to ground-based imagery.

Attention Mechanisms and Advanced Feature Extraction:

Attention mechanisms have emerged as powerful tools for enhancing feature extraction in UAV-based wildfire detection models. (Xin Chen, 2023) developed RepVGG-YOLOv7, which integrated the ECA attention mechanism and SIoU loss function to enhance feature extraction, particularly for small targets and complex backgrounds often encountered in aerial imagery. Their implementation of RepVGG in the YOLOv7 backbone enabled enhanced feature extraction during training while achieving lossless compression during inference with the model achieved an impressive 95.1% detection accuracy.

Similarly, (Soon-Young Kim, 2023) proposed a refined YOLOv7 model specifically designed for UAV imagery with the incorporation of the CBAM attention module, to enhance the feature extraction capabilities.

Their approach also included an SPPF+ layer to better concentrate on smaller wildfire smoke regions and decoupled heads with BiFPN for multi-scale feature fusion. Using a dataset of 6,500 UAV images, their model achieved 86.4% AP50, outperforming other single and multiple-stage detectors by 3.9%.
The authors noted that the addition of the CBAM attention mechanism was especially effective at distinguishing smoke plumes from visually similar features such as clouds and fog.

Limitations and Challenges:

Despite the significant advancements, several persistent challenges remain in UAV-based wildfire and smoke detection. Most studies still identified difficulties in distinguishing smoke from atmospheric phenomena such as fog, clouds, and haze. This challenge is particularly pronounced in UAV imagery, where the aerial perspective often captures cloud formations that can be visually like nascent smoke plumes.
Detection performance also typically degrades in low-light or nighttime conditions, limiting the 24-hour operational capability of these systems. Additionally, (Long Sun, 2023) noted limitations in dealing with low-resolution smoke at early fire stages, which was particularly challenging in UAV-based detection where the distance between the sensor and the target can be substantial.
Detecting small fires and minimizing false positives are key challenges in YOLO-based UAV fire detection systems. Studies highlight that baseline YOLO models struggle to detect small flames and diffuse smoke, particularly in complex scenarios captured from aerial perspectives. For example, DSS-YOLO improves upon YOLOv8n by enhancing accuracy for obscured and small fire targets, yet even this model faces difficulties under extreme conditions, such as low resolution or overexposure.
False positives present another significant issue with YOLO models deployed on UAVs. Research reveals that these models frequently classify fire-like objects such as sunlight reflections or bright surfaces as fires due to similarities in brightness and shape. This leads to false positives and reduces reliability in real-time applications. From aerial perspectives, these challenges are amplified as sun glint on water bodies, metal roofs, or other reflective surfaces can create visual patterns easily mistaken for fires.
Finally, the computational demands of deploying sophisticated deep learning models in the resource-constrained environments of UAVs remains a significant challenge. Several studies highlighted the ongoing trade-off between detection accuracy and computational efficiency. This balance is particularly critical for UAV deployments where battery life, payload capacity, and onboard computing resources are all severely limited.

2.3 Closing Remarks:

The reviewed literature demonstrates significant progress in implementing YOLO-based models for UAV-based wildfire and smoke detection. The evolution from earlier CNN approaches to specialized YOLO variants has yielded substantial improvements in both detection accuracy and computational efficiency, both critical factors for effective UAV deployment. Innovations in attention mechanisms, feature extraction techniques, and architectural optimizations have addressed many of the unique challenges posed by aerial perspectives.

However, persistent challenges related to small target detection, atmospheric interference, and computational efficiency indicate substantial opportunities for further advancement in this critical application domain.

Despite notable progress using earlier and mid-generation YOLO models, a striking gap in the literature is the lack of investigation into the capabilities of the most recent YOLO iterations, such as YOLOv11. As the YOLO architecture continues to evolve, newer versions offer substantial improvements in speed, precision, and lightweight design. Factors especially relevant to UAV-based wildfire detection, where computational efficiency and real-time response are critical. Based on the facts release by *Ultralytics*, YOLOv11 shows substantial benefits in terms of accuracy and computational strength compared to its predecessors, seen in Figure 1.

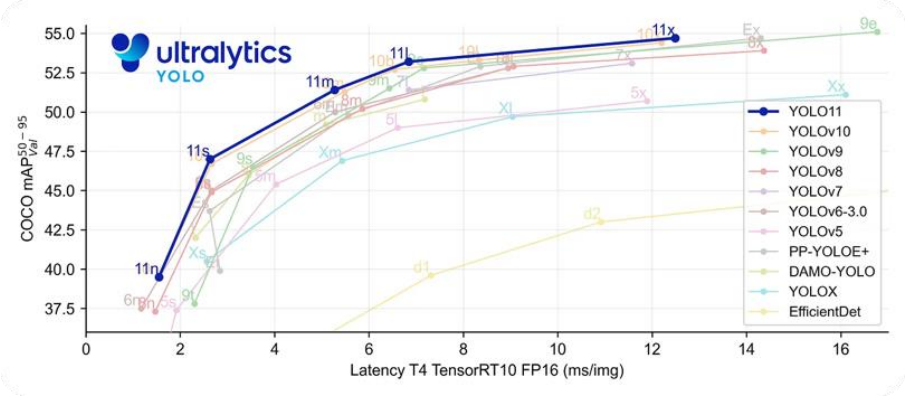

Figure 1: Comparison of Object Detection Models on COCO Validation Set: Accuracy vs. Latency (TensorRT10 FP16 on T4 GPU) (Ultralytics, 2025).

With strategic architectural modifications tailored to the challenges of aerial imagery such as enhanced attention mechanisms, specific backbone substitutions, or implementations of transfer learning techniques we could expect YOLOv11 to deliver even better performance metrics than its predecessors.

Chapter 3

Environment Setup and Models

3.1 Dataset

The D-Fire dataset was developed by (Venâncio & Gaia, 2022) to train models to detect smoke and fire. The original dataset consisting of 21,527 labelled images of fire (1164), smoke (5867), fire and smoke (4658) and with neither fire nor smoke (9838) from both ground-level and aerial images. The captured smoke and fire images in the dataset were of various forms including different shapes, colours, textures and sizes as well as being captured in different environments, such as during different times of the day, within fog and clouds, with various lighting variations and even during rain.

However, due to the computational restrictions with training such a large dataset, I decided to create a new dataset from these images to form my 'train', 'test' and 'validation' sub-sets to train my YOLO model.

The new reduced dataset called *'reduced_D-Fire'* was made from randomly chosen images from the D-Fire dataset whilst creating a balanced subset for manageability and balanced class distribution. This included each sub-set maintaining an 80:20 split of having images supporting fire and/or smoke to images not supporting fire or smoke. Further details can be seen below for each subset:

Subset of images	**Fire/smoke/ both images**	**Non-Fire images**	**Total images**
Train Dataset	3200	800	4000
Test Dataset	860	215	1075
Validation Dataset	620	155	775

Table 1: reduced_D-Fire

Each image was labelled in the YOLO format with bounding boxes for 2 classes (fire and smoke), with the non-fire images being left unlabelled. The portion of *non-fire images* in each split (approximately 20% of each set) tests the models' false-alarm rate and ensures they learn to distinguish true fire/smoke signals from background.

3.2 Hardware

For the purposes of this paper, training and testing was conducted on *'Google Colab'* using *NVIDIAs T4 and L4 GPUs*, with access to 16 GB and 22 GB of GPU VRAM respectively. These GPUs provide sufficient memory and computation for 512×512 images and large models. To ensure that all model comparisons were performed the same including inference speed measurements, I trained the models on the L4 GPU. Whilst the T4 GPU was used to test/de-bug my code during the experimentation phase.

3.3 Software

The code was implemented in PyTorch via the *Ultralytics* YOLO framework (which supports YOLOv5/7/8/11 models) and custom Python scripts for model modifications. Access to different YOLO nano models used for testing were through the official Ultralytics page and the *reduced-D-Fire* dataset was stored on my personal Google Drive.

3.4 Performance Metrics

To evaluate the fire and smoke detection models, we use the following metrics:

a. Precision

$$Precision = \frac{TP}{TP + FP}$$

b. Recall

$$Recall = \frac{TP}{TP + FN}$$

c. F1 Score

$$F1\ Score\ = 2\ \frac{Recall * Precision}{Recall + Precision}$$

d. Mean Average Precision (mAP)

$$mAP = \frac{1}{N}\sum_{c=1}^{N} AvgPrecision_c$$

N is the total number of classes
$AvgPrecision_c$ is the Average Precision for class 'c'

TP, TN, FP, FN represent True-Positive, True-Negative, False-Positive and False-Negative respectively which pertain to the Confusion Matrix shown below:

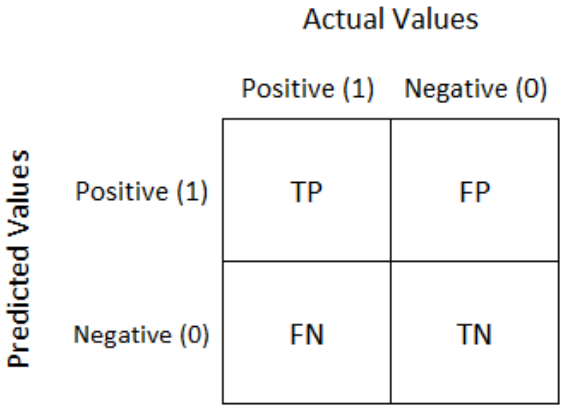

Figure 2: Confusion Matrix (Medium, 2018).

Precision reflects how many predicted fire/smoke detections are correct - measures the accuracy of positive predictions. Recall measures how many of the actual/true fire/smoke instances were detected. The F1-score (harmonic mean of precision and recall) provides a single measure for the balanced metric for both accuracy and coverage. The mean Average Precision at 50% IoU (mAP@50) is the primary metric, summarizing detection performance across both classes as the area under the precision–recall curve at an IoU threshold of 0.5.
In addition, we record inference speed (images processed per second) for each model by running the trained model on the test set and measuring throughput on the same GPU. This addresses the real-time requirement of wildfire monitoring. All metrics are computed for each model under the same conditions to facilitate direct comparison.

3.5 ML Models and Architectures:

This section details the architecture of the various models we use implement in this paper.

3.5.1 YOLOv11

YOLOv11 represents the latest advancement of the YOLO series, with improved speed, efficiency, and versatility, expanding its range of use-cases for real-time object detection.

The YOLOv11 model replaces the conventional C2f blocks with a more advanced C3k2 module, an efficient variant of CSP (Cross Stage Partial). This block implements two smaller convolutions instead of a single large one, with the 'k2' signifying the smaller kernel size, resulting in being computationally efficient without compromising detection accuracy (Khanam & Hussain, 2024).

Additionally, the inclusion of the Cross Stage Partial with Spatial Attention (C2PSA) enhances the model's ability to focus on critical spatial features within the input, refining the localisation of objects by emphasizing important areas/regions in the feature maps. Alongside these modules, YOLOv11 has also implemented a Spatial Pyramid Feature Fusion (SPFF) module, which helps with multi-scale feature aggregation in the backbone of the model (Khanam & Hussain, 2024). This allows the model to detect objects of varying sizes and positions more effectively, contributing to a higher mAP score.

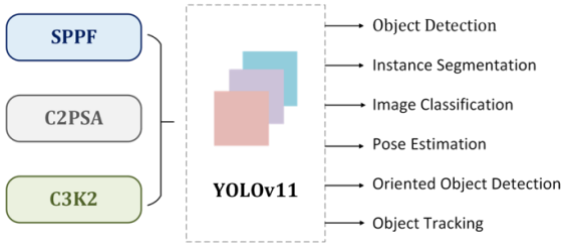

Figure 3: YOLOv11 architecture with use-cases (Gallagher, 2024).

With the paper's intent to provide a lightweight, fast, and accurate model to detect wildfires using UAV (edge device) imagery, I have decided to implement the nano version of YOLOv11 for testing and training purposes. The nano version consists of the same number of layers as the 'small' version, while having roughly 3.5 times lower number of parameters, making it faster and more lightweight for detecting wildfires. <appendix>

3.5.2 CBAM

CBAM (Convolutional Block Attention Module) is a lightweight attention mechanism designed to enhance the feature sensitivity of CNNs by focusing on the more important parts of the image.

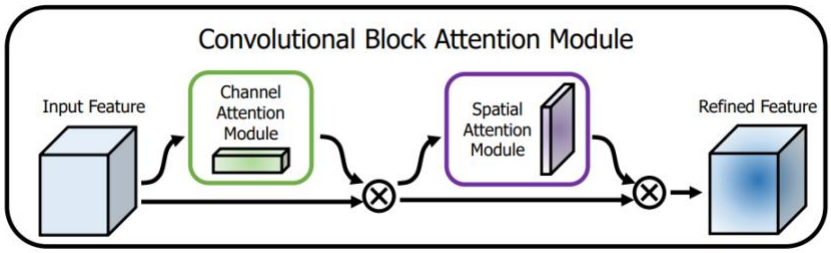

Figure 4: Architecture of the Convolutional Block Attention Module (CBAM): Sequential Application of Channel and Spatial Attention for Feature Refinement (Misra, 2024).

This is done through two main sequential steps:

a. Channel Attention Module (CAM)
b. Spatial Attention Module (SAM)

Channel Attention Module

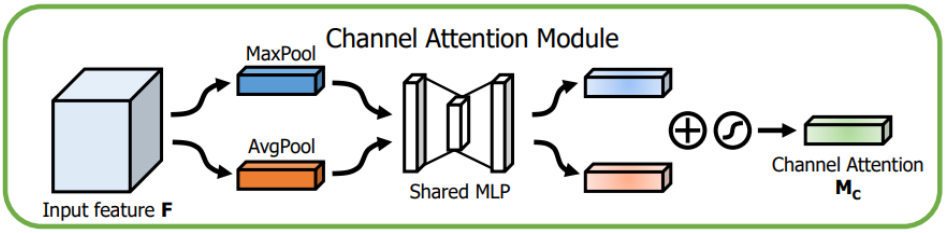

Figure 5: Aggregates Max-Pooled and Avg-Pooled Features via a Shared MLP to Compute Channel Attention Map (Misra, 2024).

Given a feature map $F \in R^{CxHxW}$, CBAM first implements the CAM to determine 'what' feature channels are the most relevant. This is achieved by applying both average pooling and max pooling across the spatial dimensions of the feature map resulting in 2 descriptors R^{Cx1x1}.

These descriptors are then forwarded through a shared multi-layer perceptron (MLP), summed and activated by a sigmoid function to produce a channel attention map M_c. The input feature map is then recalibrated by element-wise multiplication with M_c, yielding an update feature map F'.

Spatial Attention module

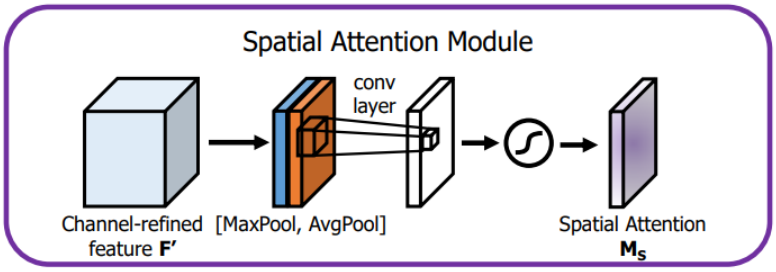

Figure 6: Structure of the Spatial Attention Module (SAM): Applies Convolution over Concatenated Max- and Avg-Pooled Features to generate Spatial Attention Map (Misra, 2024).

This feature map F' now becomes the input feature map to the next Spatial Attention Module. In this step SAM identifies 'where' the important spatial regions are in the image. It pools F' along the channel dimension using both average and max pooling, concatenates the results and applies a 7x7 convolution followed by a sigmoid function to generate a spatial attention map $M_s \in R^{1xHxW}$. This map is then applied to F' via element-wise multiplication, producing the final refined feature map F''.
Implementing these 2 steps allows CBAM to selectively highlight informative features while suppressing irrelevant ones, leading to more effective learning and better generalization. With one of CBAM's key strengths being the addition of a minimal computational overhead, this makes it suitable for integration into existing CNN architectures with real-time use cases.

Chapter 4

Experimental Hypotheses

Based on the identified research gaps, this paper proposes to test the following hypotheses:

4.1 YOLOv11 Architecture Comparison

Hypothesis 1: *YOLOv11, provides superior performance metrics (mAP, precision, recall, and inference speed) for wildfire and smoke detection compared to previous YOLO architectures when evaluated on the reduced D-Fire dataset.*

This hypothesis is supported by the Ultralytics documentation, highlighting the increased accuracy and detection scores in recent YOLO versions, along with faster performance compared to their predecessors (Ultralytics, 2025). This hypothesis also stems from analysing the evolutionary improvements in the YOLO architecture series, where each iteration typically offers enhanced feature extraction capabilities and improved handling of complex backgrounds (Gonçalves, et al., 2024). Additionally, the YOLOv11 model introduces several architectural changes that enable it to outperform prior iterations. The introduction of the 'C3k2' block leverages smaller 2x2 convolutional kernels to enhance feature extraction while reducing the computational complexity over the standard 'C3' blocks found in YOLOv5 and YOLOv8 models. Furthermore, the incorporation of the 'C2PSA' block provides a more refined attention mechanism that improves the detection accuracy, especially within occluded scenes (Khanam & Hussain, 2024).

4.2 Backbone substitutions

Hypothesis 2: *The implementation of alternative backbone architectures, such as ConvNeXt and EfficientNet, will result in a more accurate and functionally efficient model.*

Supported by (Tan, et al., 2020), EfficientNet offers significant advantages due to its efficient and scalable architecture for feature extraction. Its core strength lies in its compound scaling methods that balance network depth, width, and resolution, leading to improved performance with higher accuracy and fewer parameters. Whereas ConvNeXt-Tiny introduces modern architectural elements to convolutional networks based on Vision Transformers, such as Layer Normalisation and GELU activation functions. These enhancements are responsible for effectively capturing

global and local features, making it a strong competitor for object detection tasks.

(Zhang, et al., 2024) discuss a study that implementing ConvNeXt V2 with a YOLOv8n model for synthetic diamond quality evaluation outperforms the baseline YOLOv8n in precision, recall and mAP metrics, making it suitable for more accurate detections, while also being a lightweight module.

4.3 Attention Implementation

Hypothesis 3: *Integrating a Convolutional Block Attention Module (CBAM) into different parts of the YOLOv11 architecture; specifically, the backbone, neck or key junctions would improve detection accuracy and efficiency.*

This hypothesis investigates if implementing a Convolutional Block Attention Module (CBAM) into the various parts of the YOLOv11 architecture, specifically the backbone, neck, comprehensively throughout all or within certain layers, would improve detection accuracy and efficiency. The supporting argument states that selective attention mechanisms can enhance feature representation in object detection networks, particularly for challenging detection scenarios. This builds upon the foundational work of (Woo, et al., 2018), which demonstrates that channel and spatial attention mechanisms can significantly improve the representational power of CNNs for image detection tasks. Hence, this research extends this concept to the domain of object detection for smoke and fire detection, specifically within the YOLO architecture.

4.4 Transfer Learning with Frozen Layers

Hypothesis 4: *A two-stage transfer learning strategy with Frozen Layers would optimize the trade-off between accuracy and training cost, compared to training all layers for 200 epochs straight.*

With this hypothesis, we examine whether freezing the backbone of YOLOv11 in the initial phase of training yields efficiency benefits for wildfire detection. The rationale comes from transfer learning practices, where the backbone carries generic visual features learned from a large dataset (COCO), while the detection head is task specific. Freezing the backbone at the outset enables the model to efficiently adapt its detection layers to the specific fire and smoke classes, without altering the foundational low-level features. This approach not only accelerates the training process but also reduces computational demands. Prior research also suggests that freezing the backbone can mitigate the risk of early overfitting, particularly in cases involving limited data. Models leveraging robust pre-trained features have, in some instances, even achieved superior performance without full fine-tuning (Vasconcelos, et al., 2022).

Chapter 5

Experimentation and Design

This section details the approach and motivation behind the previously defined hypotheses.

5.1 Hypothesis 1: *YOLOv11 would outperform previous YOLO versions (YOLOv5, YOLOv8, YOLOv10) on the reduced_D-Fire dataset in terms of mAP-50, precision, recall and inference speeds.*

This section outlines a comparative experimental analysis of the 'nano' variants of YOLOv5, YOLOv8, YOLOv10, and YOLOv11 for the task of smoke and fire detection. The table below details the architectural design of each model, including the number of parameters and the measure for computational complexity in GFLOPs (Giga Floating Point Operations), which serve as indicators of memory and computational requirements for each model.

YOLO nano version	Backbone	Neck	Parameters (M)	FLOPs(B)
YOLOv11n	C3k2 + SPPF+ C2PSA	C3k2	2.6	6.5
YOLOv10n	Enhanced CSPNet	PAN	2.3	6.7
YOLOv8n	Custom CSPDarknet53 + SPPF	C2f	3.2	8.7
YOLOv5n	CSP	SPPF+PA-Net	1.9	4.5

Table 2: YOLO model architecture (Ultralytics, 2023) (Ultralytics, 2025), (Ultralytics, 2023), (Ultralytics, 2024)

All models were sourced from the official *Ultralytics* repository and were pre-trained on the COCO dataset.

Implementation Details:
This research focuses on fine-tuning, pre-trained models on the 4,000 images belonging to the *'train'* folder of the *'reduced_D-Fire'* dataset to improve their performance in detecting fire and smoke. By maintaining consistency in the dataset, training duration, and hyperparameter settings, any observed differences in performance can be attributed primarily to variations in model architecture and their respective capacities for feature extraction. The hyperparameters used for training all the models are as follows:

Device	cuda
Epochs	200
Image size (imgsz)	512
Batch size	16

Table 3: Base Hyper-parameters

5.2 Hypothesis 2: *The implementation of alternative backbone architectures, such as ConvNeXt and EfficientNet, will result in a more accurate and functionally efficient model.*
This section investigates the impact of substituting the default YOLOv11 backbone with two modern convolutional neural network architectures: *EfficientNet* and *ConvNeXt*. Both EfficientNet and ConvNeXt are recognised for their superior feature extraction capabilities, which have consistently led to substantial performance improvements across a wide range of computer vision tasks as highlighted in sections above.
The primary objective is to assess whether substituting YOLOv11's original feature extractor with these architectures enhances the detection accuracy, optimizes precision-recall trade-offs and improves functional efficiency when evaluated on the *reduced_D-Fire* dataset.

Two custom model variants are constructed for this purpose:

- YOLOv11-EfficientNet: YOLOv11 modified with an EfficientNet backbone.
- YOLOv11-ConvNeXt: YOLOv11 modified with a ConvNeXt backbone.

By isolating the effect of the backbone substitution, the study seeks to determine the extent to which modern feature extraction techniques influence overall model performance.

Implementation Details:
The backbone replacement was implemented programmatically by developing modular PyTorch components, as detailed below.

Backbone Integration:
A custom wrapper class was implemented to incorporate the EfficientNet and ConvNeXt backbones into the YOLOv11 architecture. The *EfficientNetBackbone* and *ConvNeXtBackbone* classes utilize the *'timm'* library to instantiate pre-trained models capable of producing multi-scale feature maps. These backbones modules were pre-trained on the ImageNet-1K dataset, which consists of approximately 1.2 million training images and 50,000 validation images.

In deep learning for computer vision, networks benefit from analysing images at multiple spatial scales:

a. High-resolution features (early in the network): Captures fine-grained details which include textures, edges and small objects like small smoke plumes.
b. Mid-resolution features (middle layers): Captures larger object parts like parts of a flame.
c. Low-resolution features (deeper layers): Captures global structures like the overall shape of the fire or smoke.

Extracting features across multiple resolutions is critical for object detection tasks, enabling the model to recognize both fine and coarse visual patterns. Therefore, in this implementation, the models are configured to output feature maps from multiple intermediate layers rather than solely from the final layer.

Training:
The training process involves integrating the pre-trained backbone models into the YOLOv11 architecture as substitutes for the original backbone, while preserving the neck and detection head components. To maintain a balance between computational efficiency and detection performance, the smallest variants of each backbone were selected for implementation.
Specifically, the *EfficientNet-B0* model was utilized for EfficientNet-based experiments, while the *ConvNeXt-Tiny* variant was chosen for ConvNeXt-based experiments. These lightweight configurations were selected to preserve the real-time inference capabilities and low computation overhead

that are critical for the intended smoke and fire detection application. Other model versions can be seen in the appendix A.1.
Both models were then trained and tested on the *reduced-D-Fire* dataset with the standard hyper-parameters as listed above.

5.3 Hypothesis 3: *Integrating a Convolutional Block Attention Module (CBAM) into different parts of the YOLOv11 architecture; specifically, the backbone, neck or key junctions would improve detection accuracy and efficiency.*

Implementation Details:
The CBAM architecture is implemented as a modular enhancement that can be selectively integrated into specific layers of the YOLOv11 model. The implementation follows the original design described by (Woo, et al., 2018) with several adaptations specific to the YOLOv11 architecture. These adaptations were necessary to ensure compatibility and optimal performance when integrating attention mechanisms into a detection network rather than a classification network as in the original paper.

Channel Attention Module
The implementation adopts the dual-pathway design proposed by (Woo, et al., 2018), but incorporates additional adaptive reduction ratios to accommodate the varying channel dimensions present throughout the YOLOv11 architecture.
The adaptive reduction ratio implementation was taken from (Wang, et al., 2020), who demonstrated that fixed reduction ratios can be suboptimal when working with networks that have varying channel dimensions at different depths. Their work showed that smaller feature maps with fewer channels require more conservative reduction to prevent information bottlenecks, while larger feature maps can utilize more aggressive reduction without significant information loss.
Another further addition to the Channel Attention Module would be the implementation of the *weight initialization strategy* that employs a small scaling factor of 0.1 supported by (Wadii Boulila, 2024). The respective paper demonstrated that initializing attention modules with smaller weights can improve training stability and convergence when integrating them into pre-trained networks. This approach allows the network to gradually adapt to the attention mechanism rather than causing dramatic shifts in feature representation early in training.

Spatial Attention Module

The Spatial Attention Module identifies regions of interest within feature maps by generating a spatial attention map. As recommended by (Woo, et al., 2018).

The innovation approach in this implementation lies in the adaptive kernel size selection. Research by demonstrated that different network stages benefit from different receptive field sizes in attention mechanisms (Wei, et al., 2017). Early layers benefit from larger kernels to capture broader spatial context, while later layers achieve better performance with smaller kernels that can focus on refined features.

Integrated CBAM Module

The complete CBAM module integrates both attention mechanisms with position-aware parameter adjustments. The sequential application of channel attention followed by spatial attention follows the optimal arrangement identified by (Woo, et al., 2018).

The addition of a learnable attention scale parameter represents an enhancement to the original CBAM formulation. This approach draws inspiration from (Wang, et al., 2020), who demonstrated that allowing the network to learn the appropriate influence of attention mechanisms can lead to more stable training dynamics and better overall performance.

The attention scale acts as a gating mechanism that can modulate the impact of attention during training.

Integration Strategy Design

The integration of CBAM modules into YOLOv11 follows a systematic approach with five distinct strategies, each targeting different aspects of network enhancement. This comprehensive exploration of integration strategies is motivated by research from (Yue Cao, 2019) who demonstrated that attention mechanisms can have varying effects depending on their placement within detection architectures.The implementation identifies appropriate integration points through structural analysis of the model.

This function establishes the boundaries between model sections based on architectural knowledge of YOLOv11's structure. The division between backbone and neck is consistent with the architectural design principles outlined in recent literature on the YOLO variants.

Layer Position Adaptation
To optimize CBAM for different network positions, kernel sizes are adapted based on the layer's location. This position-aware adaptation is informed by empirical findings from (Li, et al., 2019), demonstrated the importance of receptive field optimization across different network depths. The early backbone layers process low-level features with broader spatial context, necessitating larger kernels (7×7), the late backbone layers handle intermediate features requiring medium receptive fields (5×5), while the neck layers, which process refined semantic information, benefit from smaller kernels (3×3) that can focus on precise feature localization.

Target Layer Selection and Strategy Implementation
The selection of layers to enhance with CBAM follows a structured approach that identifies appropriate integration points based on the experimental strategy. This recursive pattern allows for comprehensive mapping of the model architecture, creating a detailed representation of potential integration points with their associated metadata. The channel threshold filtering (layers with at least 32 channels) is informed by principles from (Hu, et al., 2017) and (Woo, et al., 2018), which demonstrates that attention mechanisms require sufficient channel dimensionality to be effective, with diminishing returns observed when applied to feature maps with very few channels. This threshold ensures computational efficiency while maintaining the benefits of attention enhancement.
The implementation supports *five distinct integration strategies*, each with a specific theoretical motivation and with practical objective detailed below:

5.3.1 Backbone-Only Integration
The backbone-only strategy focuses attention enhancement on the feature extraction component of the network. This approach is motivated by research from (Lee & Chung, 2021), who demonstrated that enhancing feature extraction in early network stages can significantly improve the quality of base features propagated throughout the network.
By implementing computational resources on the backbone, this strategy aims to enhance the feature extraction quality without increasing computational burden in the detection head. This approach offers several additional benefits including improved robustness to occlusion (Zhang, et al., 2020), better low-level feature extraction (Liang, et al., 2021), and reduced sensitivity to background clutter (Cao, et al., 2019).
Furthermore, attention-enhanced backbones produce feature maps with

better boundary definition, leading to more precise localization capabilities (Woo, et al., 2018), while also increasing the discriminative power of features propagated to later network stages (Chen, et al., 2020). For detection tasks specifically, (Li, et al., 2019) demonstrates that backbone feature quality significantly impacts detection performance bounds, with their research showing that enhancements to backbone architecture yield consistent improvements across multiple detection frameworks. Similarly, (Tan, et al., 2020) in their *EfficientDet* work quantified the relative importance of backbone, neck, and detection head components, finding that backbone enhancements provided the most substantial performance gains for a given computational budget.

5.3.2 Neck-Only Integration

The neck-only strategy applies attention mechanisms exclusively to the feature pyramid network (FPN) component responsible for multi-scale feature fusion. This approach draws on findings from (Tan, et al., 2020), which showed that attention mechanisms in the neck components can enhance cross-scale feature selection. This is particularly beneficial for detecting objects at varying scales. By focusing on the neck, this strategy aims to improve the model's ability to handle scale variations while maintaining computational efficiency. Additional benefits of neck-focused attention include improved feature alignment across scales (Ghiasi, et al., 2021), enhanced contextual information selection and more effective suppression of false positives in dense prediction scenarios (Cao, et al., 2019).
(Cao, et al., 2019) demonstrated that attention in the neck components specifically helps balance the influence of features from different backbone levels, preventing deeper features from dominating the detection process. Furthermore, attention-enhanced necks improve information flow between high-resolution and low-resolution feature maps, enabling better small object detection without requiring computationally expensive high-resolution backbone features. This targeted enhancement approach provides an effective balance between performance improvements and computational overhead.

5.3.3 Comprehensive Integration

The ‘all’ integration strategy applies CBAM modules throughout the entire network, that is after each layer in the neck, head and backbone. This approach follows the principle articulated by (Woo, et al., 2018) that attention benefits can accumulate across network depths, building on the CBAM framework for modular attention integration. While the only drawback being computationally more intensive, this strategy tests the hypothesis that comprehensive attention integration provides additive benefits not achievable.

5.3.4 Domain-Specific Wildfire Detection Strategy

This strategy is informed by research from (Zhang, et al., 2021), who demonstrated that wildfire detection presents unique challenges requiring attention to both texture and colour features at multiple scales. The strategy also hopes to reduce the computational overhead compared to the ‘Comprehensive Integration strategy’ whilst retaining/improving the accuracy. The selection of specific layers follows a principled approach based on the feature hierarchy established by Gao et al. (2023) for fire detection:

1. *Early Backbone Layers* (models.2, model.4): These layers capture low-level features such as edges, textures, and colour transitions that are crucial for smoke detection. (Zhang, et al., 2021) demonstrated that smoke presents as diffuse patterns with gradual transitions, which are best captured in early convolutional layers. Enhancing attention in these layers can improve sensitivity to the subtle, diffuse patterns characteristic of smoke.
2. *Mid Backbone Layer* (model.6): This layer operates at an intermediate level of abstraction, capturing structured patterns that correspond to flame characteristics. As established by (Ryu & Kwak, 2022), flame detection benefits from features that capture both colour intensity and structural patterns, which are well-represented at this network depth.
3. *Late Backbone and Neck Integration* (model.9, model.13): These layers integrate regional information across spatial scales. (Shang-Hua Gao, et al., 2021) demonstrated that contextual integration is particularly important for distinguishing wildfires from visually similar non-fire phenomena. Enhancing attention at these junction points can improve the model's ability to leverage contextual cues for accurate fire/non-fire discrimination.

4. *Final Feature Representation* (model.17): This layer prepares features for final detection task. Adding attention here can enhance the model's ability to focus on the most discriminative fire-related features before final detection outputs, as demonstrated by (Jadon, et al., 2019) in their work on attention-enhanced fire detection.

5.3.5 Minimal Efficiency-Oriented Strategy

The minimal strategy represents an efficiency-oriented approach that targets only critical network junctions. This highly selective approach is grounded in research based on (Wang, et al., 2020) and (Hu, et al., 2017), who investigated the efficiency-performance trade-offs in attention-enhanced networks. Their work demonstrated that while the '*Comprehensive Attention Integration'* provides the highest theoretical performance ceiling, strategic placement adopted at key network layer can capture a significant portion of the benefits with minimal computational overhead. The minimal strategy specifically targets two critical points: the backbone-neck transition (model.9) and the final feature integration layer (model.17). These junction points act as information bottlenecks where feature refinement yields disproportionately large performance gains. (Woo, et al., 2018) quantified this effect in their original CBAM paper, showing that attention at strategic locations can provide substantial performance improvement while minimizing computational cost. (Shang-Hua Gao, et al., 2021) further validated this approach, demonstrating that feature enhancement at key transition points effectively addresses both representational quality and computational efficiency requirements.

This minimalist approach is particularly valuable for deployment scenarios with strict computational constraints or real-time requirements, as it optimizes the efficiency-performance trade-off by focusing enhancement resources precisely where they deliver maximum impact

CBAM Integration Mechanism

The actual integration of CBAM modules is performed through a wrapping class that preserves the original network functionality while adding the attention mechanism. An additional adaptive reduction ratio implementation scales the channel compression based on the layer dimensions. Similarly to the testing of the other hypotheses, testing of each of the 5 CBAM implementations, keep the same common hyper-parameters as listed above.

5.4 Hypothesis 4: *A two-stage transfer learning strategy with Frozen Layers would optimize the trade-off between accuracy and training cost, compared to training all layers for 200 epochs straight.*

Building upon the results from the previous hypothesis, it is evident that incorporating CBAM into the YOLOv11 model yields consistent improvements across key performance metrics, including accuracy, precision, recall, and mAP@50. These findings support the decision to explore the integration of transfer learning in conjunction with the *Domain-Specific Wildfire Detection Strategy* with hopes to achieve the best result. In accordance with the Ultralytics documentation, a two-stage training schedule can be applied to the detection model as follows (Ultralytics, n.d.):

Stage 1 (Frozen-Backbone Training): During the first 100 epochs, the model was trained with the backbone layers frozen, allowing only the neck and head layers responsible for feature selection and bounding box prediction to be updated. Backbone freezing was achieved by setting '*requires_grad=False'* for all backbone parameters, utilizing the built-in functionality provided by YOLO (Ultralytics, n.d.).
A slightly reduced learning rate at 0.001 was adopted during this phase, reflecting the limited number of trainable parameters. The objective of this stage was to enable the model to effectively classify and localize fire and smoke using pre-learned features. Since only the neck and head components (approximately one-third of the architecture) were subject to gradient updates, training was computationally efficient.

Stage 2 (Fine-Tune Unfrozen): Following the initial 100 epochs, the backbone was unfrozen, allowing all layers of the YOLOv11 model to be trainable for the remainder of the training cycle (up to a total of 200 epochs). The final weights from Stage 1 'last.pt' were used to initialize this phase, ensuring continuity in learning. A reduced learning rate at 0.0001 was applied to facilitate gradual fine-tuning of the previously frozen layers. This step enables the model to adapt the pre-trained backbone to the domain-specific characteristics of wildfire imagery, which often deviates from the generic content of datasets like COCO.

Testing

At the conclusion of Stage 2, we evaluated the performance metrics of the final last.pt model and compared them against the baseline metrics. To further investigate the impact of different epoch distributions across training stages, two additional experimental configurations were introduced. In the second test, Stage 1 (frozen backbone) was limited to 50 epochs, followed by 150 epochs of full fine-tuning in Stage 2. In the third test, Stage 1 was extended to 75 epochs, with the remaining 125 epochs allocated to Stage 2.

The objective of these variations was to determine the optimal point at which freezing the backbone ceases to offer a performance advantage, thereby informing best practices for applying transfer learning in wildfire detection contexts. The table below highlights the hyper-parameters used in each stage:

Stage 1		**Stage 2**	
Device	cuda	**Device**	cuda
Epochs	Frozen number	**Epochs**	(200 – Frozen number)
Image size (imgsz)	512	**Image size** (imgsz)	512
Batch size	16	**Batch size**	16
lr0 (initial learning rate)	0.001	**lr0** (initial learning rate)	0.0001
lrf (final learning rate)	0.01	**lrf** (final learning rate)	0.001

Table 4: Hyper-parameters for transfer learning

5.5 Fair Comparison Controls for all hypotheses:

Metrics Collected:
After training, each model is evaluated on the common test set of 1,075 images. We compute precision, recall, mAP@50, F1, and log the inference speed in frames per second (FPS) on the Colab GPU.
For inference speed, each model processes the entire test set in evaluation mode and the average time per image (or FPS) is recorded. This ensures that speed is measured in a realistic scenario. We also note each model's parameter count and model size, as these factors relate to speed and can contextualize differences, though they are not the primary metrics, they help ensure fairness.

Control Variables:
To isolate the impact of model architecture, rigorous measures were taken to prevent confounding variables from influencing the results. All models were trained using the same set of images and identical augmentation strategies. Additionally, all training procedures were executed within the same computational environment to eliminate variability due to hardware or software differences. Specifically, all models are trained on an NVIDIA L4 GPU with 22 GB of VRAM.
Our objective is to allow each model to perform under conditions that align with optimal settings reported in the literature. Each model undergoes an independent training run, and convergence is verified by monitoring validation mean Average Precision (mAP) curves to ensure that none of the models are under-trained or overfitted.
In summary, this experimental setup enables a direct evaluation of whether YOLOv11's architectural improvements lead to superior detection accuracy, which is measured by higher mAP@50 and F1 scores, along with precision-recall trade-offs while still maintaining competitive real-time inference speed relative to previous YOLO versions.

Chapter 6

Results and Discussion

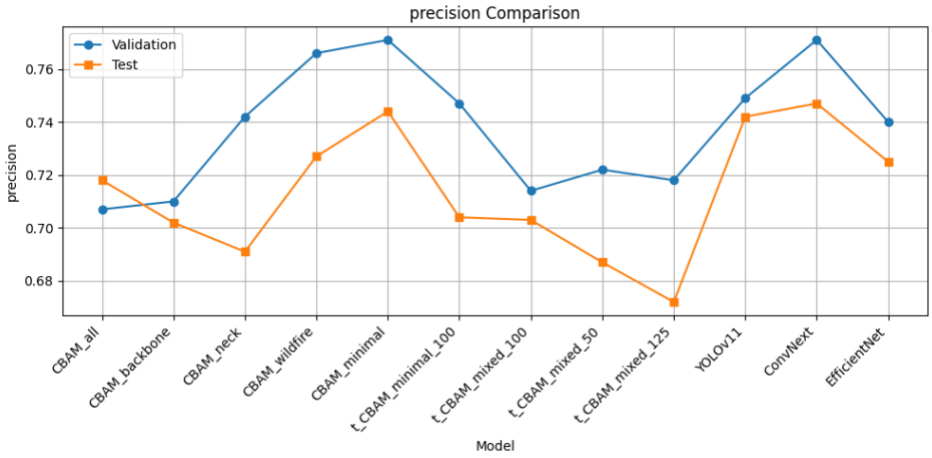

Figure 7: Precision Vs Model graph for Validation and Test Dataset

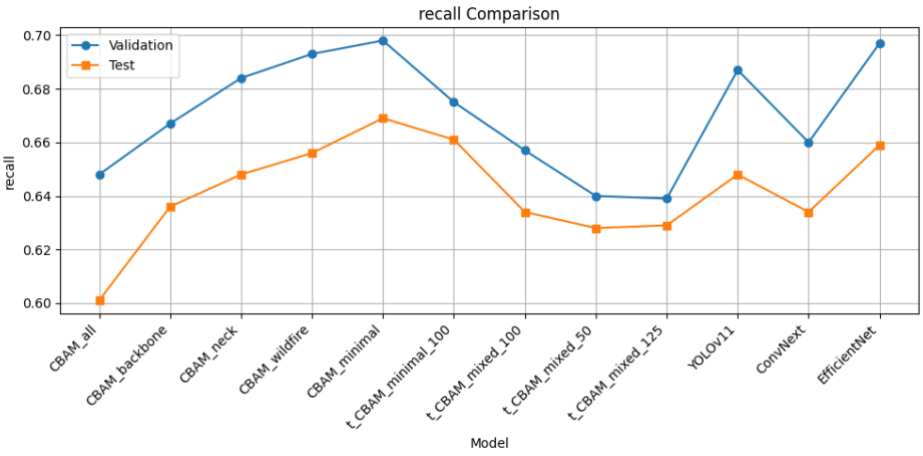

Figure 8: Recall Vs Model graph for Validation and Test Dataset

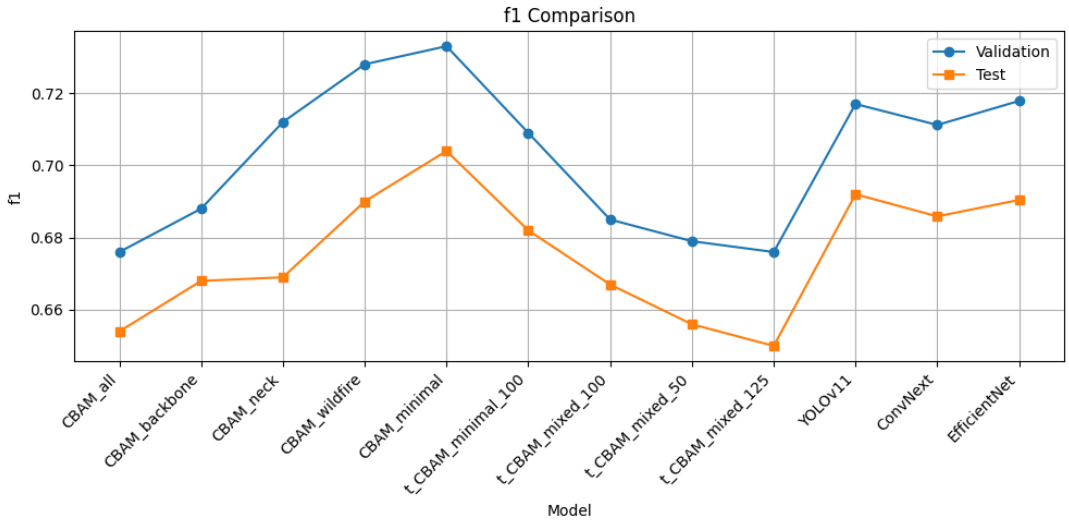

Figure 9: F1 Vs Model graph for Validation and Test Dataset

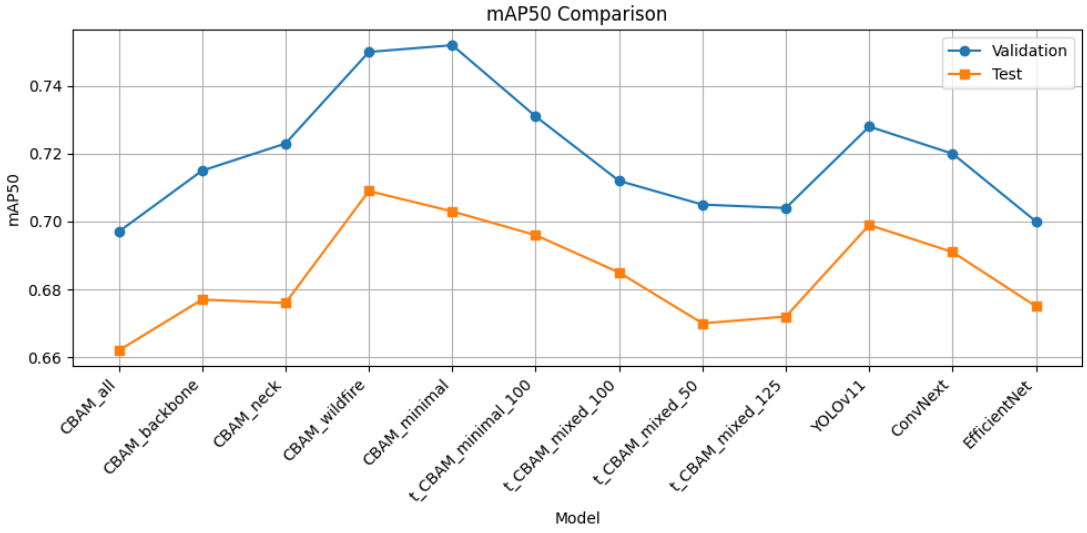

Figure 10: mAP50 Vs Model graph for Validation and Test Dataset

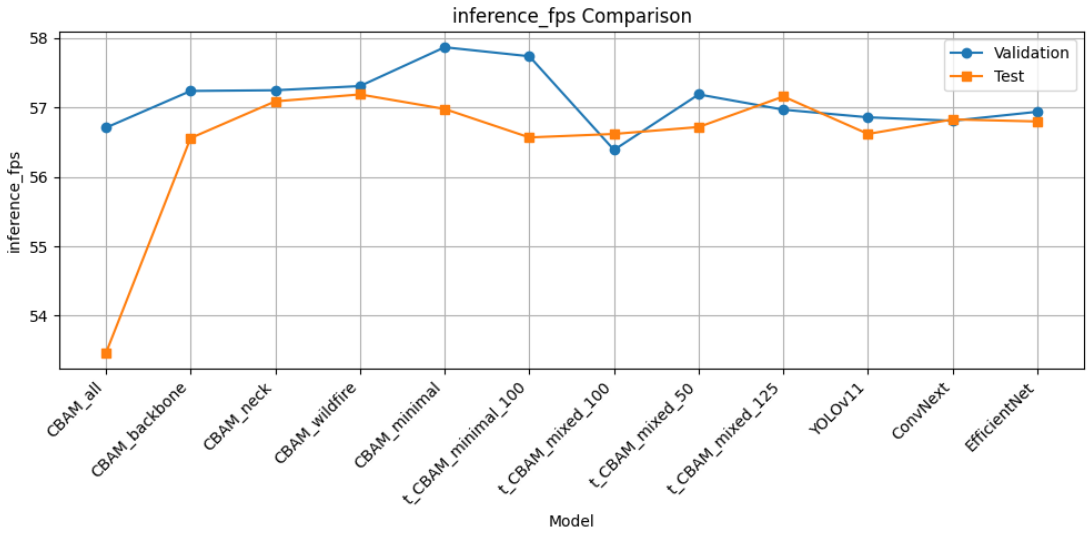

Figure 11: Inference speed (FPS) Vs Model graph for Validation and Test Dataset

model	precision	recall	f1	mAP50	inference_fps
CBAM_all	0.718	0.601	0.654	0.662	53.46
CBAM_backbone	0.702	0.636	0.668	0.677	56.56
CBAM_neck	0.691	0.648	0.669	0.676	57.09
CBAM_wildfire	0.727	0.656	0.69	**0.709**	**57.19**
CBAM_minimal	0.744	**0.669**	**0.704**	0.703	56.98
t_CBAM_minimal_100	0.704	0.661	0.682	0.696	56.57
t_CBAM_mixed_100	0.703	0.634	0.667	0.685	56.62
t_CBAM_mixed_50	0.687	0.628	0.656	0.67	56.72
t_CBAM_mixed_125	0.672	0.629	0.65	0.672	57.16
YOLOv11	0.742	0.648	0.692	0.699	56.62
ConvNext	**0.747**	0.634	0.68588	0.691	56.83
EfficientNet	0.725	0.659	0.69043	0.675	56.8

Figure 12: Mode testing on Test Dataset

The performance metrics in this section are drawn from rigorous testing on the ‘test’ and ‘validation’ dataset belonging to the *reduced_D-Fire* test dataset. All models were evaluated under identical conditions using precision, recall, F1-score, mAP@50, and inference speed (FPS) as the primary metrics. These metrics directly reflect the goals set out in the methodology—assessing both the detection accuracy and real-time performance necessary for UAV-based wildfire detection.

In this Chapter we would be investigating the metrics further with respect to our hypotheses.

Hypothesis 1:

model	precision	recall	f1	mAP50	Inference (FPS)
YOLOv11	**0.742**	0.648	**0.692**	**0.699**	54.48
YOLOv10	0.717	0.64	0.676	0.679	57.91
YOLOv8	0.734	0.643	0.685	0.695	**65.1**
YOLOv5	0.711	**0.67**	0.69	0.69	61.77

Table 5: Comparative Analysis of various YOLO models

A comparative evaluation of the metrics found in Table 5 reveals that the 'nano' versions of the YOLO architectures on the reduced D Fire dataset demonstrate that YOLOv11n offers the best balance of convergence speed, detection accuracy, and computational efficiency.

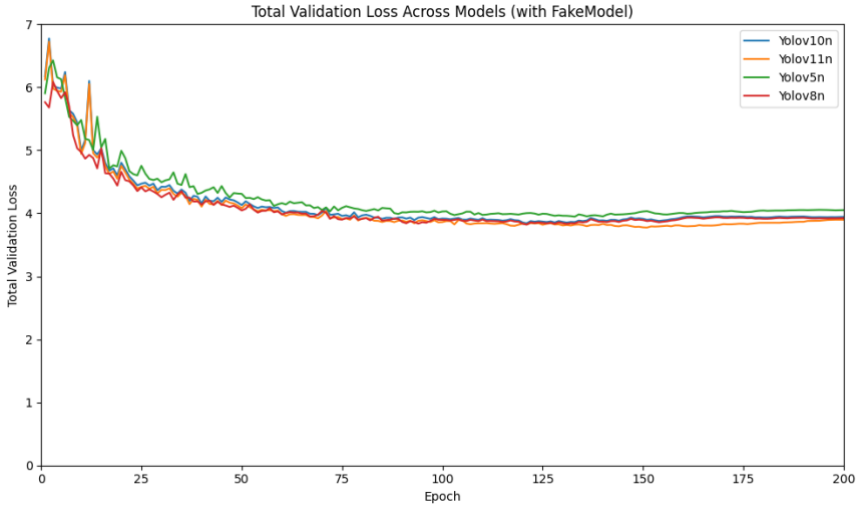

Figure 13: Total Validation Loss Vs Epoch

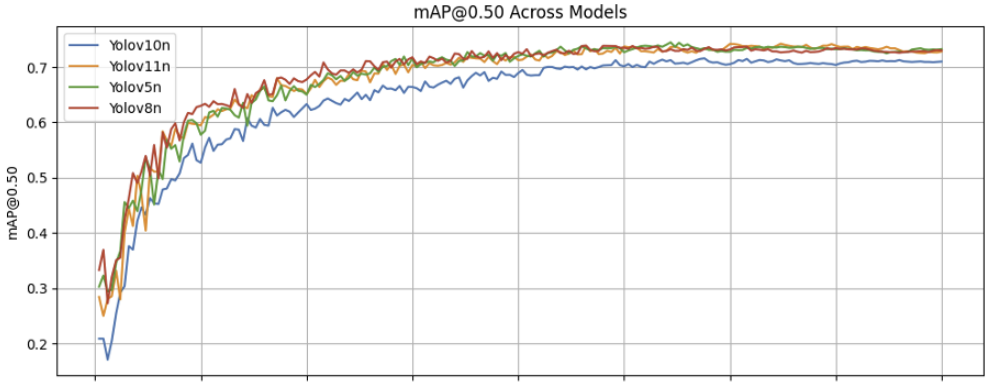

Figure 14: mAP50 Vs Epoch

During 200 epochs of training, YOLOv11n stabilised its loss by epoch 50—faster than both YOLOv5n and YOLOv10n as seen in Figure 13. It also achieved the highest single-threshold mAP@50 of 0.742, surpassing YOLOv8n, YOLOv5n and YOLOv10n, thanks to a superior precision–recall trade-off on the *fire* and *smoke* classes.

Inference speed correlates with each model's computational overhead: YOLOv11n (2.6 M parameters, 6.5 GFLOPs) sits between YOLOv5n (1.9 M, 4.5 GFLOPs) and YOLOv8n (3.2 M, 8.7 GFLOPs). Recording the highest Precision, F1 and mAP@50 values—and the second-highest Recall after YOLOv5n—YOLOv11n confirms our hypothesis that it outperforms its predecessors on reduced D-Fire, delivering the best compromise between accuracy and efficiency.

Hypothesis 2:

model	precision	recall	f1	mAP50	inference_fps
YOLOv11	0.742	0.648	**0.692**	**0.699**	56.62
ConvNeXt	**0.747**	0.634	0.68588	0.691	**56.83**
EfficientNet	0.725	**0.659**	0.69043	0.675	56.8

Table 6: Evaluation metrics against 'test' dataset

Our investigation of Hypothesis 2 examined whether replacing YOLOv11's standard backbone with either ConvNeXt-Tiny or EfficientNet-B0 could enhance YOLOv11's performance metrics. Through this comparative analysis using the 'test' and 'validation' dataset of the *reduced_D-Fire dataset,* we evaluated metrics behind three distinct configurations. These included: the baseline YOLOv11 model, YOLOv11 integrated with EfficientNet-B0, and YOLOv11 incorporating the ConvNeXt-Tiny architecture. This methodical approach allowed us to quantify performance variations across these architectural modifications.

Performance Overview:

- **YOLOv11 (Baseline):**
  - *Validation:* mAP@0.5 = 0.728, F1 = 0.717
  - *Test:* mAP@0.5 = 0.699, F1 = 0.692
- **YOLOv11-EfficientNet:**
  - *Validation:* mAP@0.5 = 0.687, F1 = 0.689
  - *Test:* mAP@0.5 = 0.664, F1 = 0.662
- **YOLOv11-ConvNeXt:**
  - *Validation:* mAP@0.5 = 0.701, mAP@0.5:0.95 = 0.377, F1 = 0.698
  - *Test:* mAP@0.5 = 0.671, mAP@0.5:0.95 = 0.358, F1 = 0.668

These results showed that neither of the backbone replacements outperformed the baseline. ConvNeXt-Tiny showed closer performance to YOLOv11, particularly in validation metrics, but both it and EfficientNet-B0 fell short in precision and mean average precision across both datasets.

There are several likely reasons for this underperformance. EfficientNet and ConvNeXt were originally designed for classification tasks (Tan & Le, 2019) and integrating them directly into a detection-focused architecture like YOLOv11 can lead to architectural mismatches. Furthermore, EfficientNet's compound scaling method, while effective in reducing parameters and improving classification accuracy, can produce feature maps with resolutions or semantic depth that are suboptimal for downstream object detection layers. Similarly, ConvNeXt's design incorporates transformer-like components such as GELU activation and Layer Normalization, which can affect feature compatibility within convolutional detection heads (Dai, et al., 2022).

Moreover, the resolution and dimensionality of the features extracted by these backbones may not align with YOLOv11's neck and head, which were designed to handle specific multi-scale inputs. In EfficientDet, for example, EfficientNet is only effective when paired with a BiFPN (Bidirectional Feature Pyramid Network), which enables effective multi-scale feature fusion (Tan & Le, 2019). This kind of adaptation was not introduced in the present implementation, possibly limiting the effectiveness of both EfficientNet and ConvNeXt in the object detection context.

The complexity of the reduced_D-Fire dataset also presents a challenge. It includes small, occluded, and irregularly shaped objects like smoke and fire plumes. The original YOLOv11 backbone, being developed and tuned for real-time object detection, appears better equipped to handle such variability than classification-optimized backbones without further tuning.

Conclusion:
The results do not support Hypothesis 2. Despite their strengths in classification, EfficientNet and ConvNeXt did not improve detection accuracy when directly integrated into YOLOv11. These findings suggest that altering the backbone of an object detector requires holistic architectural adjustments, particularly to the neck and detection head to accommodate changes in feature representations.

Hypothesis 3:

model	precision	recall	f1	mAP@50	Inference (FPS)
CBAM_all	0.718	0.601	0.654	0.662	53.46
CBAM_backbone	0.702	0.636	0.668	0.677	56.56
CBAM_neck	0.691	0.648	0.669	0.676	57.09
CBAM_wildfire	0.727	0.656	0.69	**0.709**	**57.19**
CBAM_minimal	**0.744**	**0.669**	**0.704**	0.703	56.98
YOLOv11	0.742	0.648	0.692	0.699	56.62

Table 7: Metric data from evaluating on the test dataset

In this section we evaluated the YOLOv11 model against five CBAM-enhanced variants on both validation and test set seen on Table 7. On the test set, YOLOv11 achieved precision 0.742, recall 0.648, F1 0.692, mAP@0.5 0.699 at 56.62 FPS. Its modified counterparts showed varying trade-offs. The CBAM_minimal model attained the highest overall accuracy: on test it reached precision 0.744, recall 0.669, F1 0.704 and mAP@0.5 0.703; on validation it was 0.771, 0.698, 0.733 and 0.752 respectively. Its inference speed remained high at about 57 FPS on both the ‘training’ and ‘test’ dataset. Meanwhile, the CBAM_wildfire variant also performed strongly, with it test metrics (precision 0.727, recall 0.656, F1 0.690, mAP50 0.709 at a steady FPS ~57.2) and its validation metrics (0.766, 0.693, 0.728, 0.750, FPS 57.3) very close to CBAM_minimal.

By contrast, *CBAM_all*, which inserts CBAM at every applicable layer performed the worst. On test it yielded only 0.718 precision, 0.601 recall, F1 0.654 and mAP50 0.662, noticeably below baseline YOLOv11n. Its validation scores (0.707, 0.648, 0.676, 0.697) were similarly the lowest with respect to other models too. Moreover, CBAM_all ran significantly slower at 53.5 FPS due to the added attention overhead.

Intermediate variants such as CBAM_backbone and CBAM_neck yielded modest improvements over baseline: each had slightly higher recall or precision but only marginal mAP gains and similar FPS.

Overall, CBAM_minimal delivered the best trade-off. Its selective attention modules boosted feature focus without excessive overhead, yielding the highest F1 scores of all models on both splits. The strong performance of CBAM_wildfire similarly shows that targeted attention (in the “wildfire” component) can improve smoke/fire feature learning.

In both cases, the mAP@0.5 scores approached or slightly exceeded the baseline (e.g. 0.709 for CBAM_wildfire vs 0.699 for YOLOv11 on test) even as inference speed remained comparable. These gains aligned with prior findings that well-placed CBAM modules can enhance detection by highlighting relevant spatial and channel information. For instance, (Xu, et al., 2024) showed that adding CBAM into a YOLO-based fire-detection network improved accuracy by ~2–3%, consistent with our observation that modest CBAM use benefits smoke detection.

By contrast, CBAM_all underperformed, demonstrating that over-integration of attention can backfire. The degraded mAP and F1 suggest that inserting CBAM everywhere over-regularized the small YOLOv11 network. Instead of refining features, too much attention can suppress useful signal and even amplify irrelevant patterns (Wenguang Tao 1, et al., 2024). Indeed, Wu et al. (2025) and others have noted that excessive attention modules can confuse a network's focus in limited-capacity models. In our experiments, CBAM_all's slower speed and lower accuracy reflect this: the network likely became bloated and its feature maps "over-smoothed" by attention gating. In effect, CBAM_all may have introduced feature suppression and false-positive emphasis, as seen in studies where attention indiscriminately weights background or occluded regions.

In summary, the results support Hypothesis 3 only when CBAM is applied judiciously. Limited CBAM integration (CBAM_minimal or CBAM_wildfire) yielded the best performance, improving on YOLOv11 without a speed penalty. This outcome matches the literature on lean attention modules: for example, (Wang et al. 2020) proposed the lightweight ECA module to gain attention benefits with minimal cost. Conversely, the CBAM_all variant's poor results underscore the drawbacks of "over-attention" in a small model: It suffered reduced accuracy and slower inference. Thus, our findings, that targeted CBAM placement outperforms full integration, align with published observations about over-regularization and feature confusion from excessive attention.

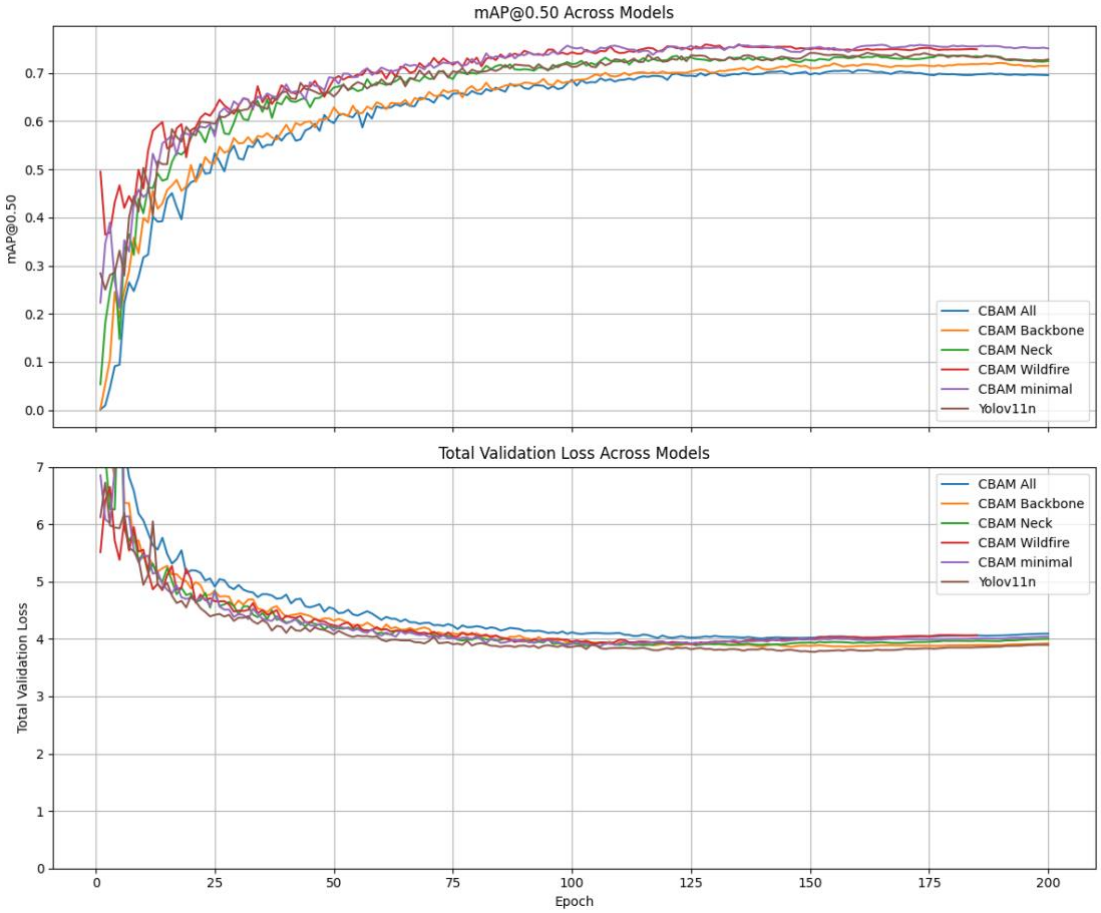

Figure 15: Evaluation metrics against 'test' dataset

Hypothesis 4:

model	precision	recall	f1	mAP50	inference_fps
t_CBAM_minimal_100	0.704	**0.661**	0.682	0.696	56.57
t_CBAM_mixed_100	0.703	0.634	0.667	0.685	56.62
t_CBAM_mixed_50	0.687	0.628	0.656	0.67	56.72
t_CBAM_mixed_125	0.672	0.629	0.65	0.672	**57.16**
YOLOv11	**0.742**	0.648	**0.692**	**0.699**	56.62

Table 8: Evaluation 'test' dataset metrics against transfer learning models

The fully trained YOLOv11 baseline achieved the highest detection metrics. For example, the *t_CBAM_minimal_100* model reached precision 0.704 and F1 0.682 (versus 0.742 and 0.692 for the baseline), with only a slight recall gain (0.661 vs 0.648). The other two-stage variants performed worse as seen in Table all trailed the baseline (P=0.742, R=0.648, F1=0.692, mAP@0.5=0.699). All models ran at similar inference speed (~56–57 FPS), so the two-stage scheme offered no throughput benefit. Overall, no two-stage model exceeded the baseline's accuracy.

These findings indicate that freezing the backbone did not improve the accuracy–efficiency trade-off. A likely cause is under-adaptation: the frozen pretrained features may not capture the specialized patterns in the target data, limiting the model's ability to learn new object feature. In particular, when source and target domains differ (e.g. general vs small-object tasks), naive transfer can even hurt performance (Xu, et al., 2023). (Sohaib et al. 2024) similarly note that generic pretraining often "fails to equip the model with the specific features" needed for subtle target objects. Freezing early layers may have over-constrained the network: it could be overfit to fixed features or simply lack sufficient gradient updates to adjust to the new domain. Indeed, Kim et al. report that a fully frozen backbone yields "considerable degradation in model performance" compared to full training consistent with our observations.

In summary, Hypothesis 4 is not supported: all two-stage variants delivered F1 and mAP slightly below the baseline, despite being close. The fully fine-tuned YOLOv11 remained superior in accuracy, while any training-speed gains did not translate to better metrics.

Chapter 7

Conclusions

This study addressed the critical challenge of early wildfire detection using UAV imagery by developing and improving YOLOv11, a state-of-the-art object detection model. Wildfires pose an increasing threat to ecosystems and communities worldwide, and UAVs equipped with automated vision can provide rapid monitoring to support firefighting efforts. The original motivation was to leverage deep learning to enhance real-time detection of fires and smoke in aerial images. By building on the latest YOLO architecture and incorporating advanced modules, the project aimed to push detection performance beyond current standards.

Summary of Achievements

The enhanced YOLOv11 model demonstrated significant performance gains over previous YOLO variants. Through systematic architectural improvements, the model achieved higher detection accuracy and robustness. Incorporating novel modules such as C3k2, C2PSA, and SPFF enriched the network's feature extraction and multi-scale fusion capabilities. The C3k2 blocks, which use smaller convolutional kernels, improved processing speed without sacrificing detection performance. The C2PSA units combined cross-stage partial connections with spatial attention to preserve contextual information across layers.

In parallel, attention mechanisms and modern backbones were integrated. The Convolutional Block Attention Module (CBAM) was added to guide the model's focus on relevant spatial and channel information, highlighting subtle fire or smoke signals. Replacing the standard YOLO backbone with EfficientNet and ConvNeXt variants further boosted feature representation. These backbones are known for their strong performance-to-complexity ratio, and in this project they provided richer, more discriminative features for identifying wildfire indicators.

Ablation studies confirmed the positive impact of each enhancement, and qualitative evaluations on drone imagery showed that the improved model could detect early-stage fires and smoke plumes that simpler models often missed. Collectively, these outcomes demonstrate that the combined enhancements resulted in a model well-suited for the demanding task of early wildfire detection.

Challenges and Limitations

Despite these successes, the project encountered several challenges. One persistent difficulty was detecting very small fire targets in high-altitude images. Early fire outbreaks often occupy just a few pixels, making them easy to overlook. Although the model's enhancements helped, there were still cases where faint flames were missed or only partially localized. Closely related to this was the issue of false positives from non-fire phenomena. The model sometimes confused smoke or haze for low-altitude clouds or fog, reflecting the inherent ambiguity between smoke and other atmospheric particles in 2D imagery.

Another significant challenge was balancing inference speed with accuracy. Each added module or attention block improved detection quality but also increased computational load. Maintaining real-time processing on a UAV's limited hardware required careful optimization. For example, the C3k2 blocks helped mitigate some of the speed penalty, but when combining multiple enhancements (such as CBAM, deeper backbones, and SPFF), inference became slower. It was necessary to make trade-offs between further accuracy improvements and the practical requirements of deployment.

The development process also faced practical limitations. The training dataset was relatively small and less varied than ideal. Because annotated wildfire images from UAVs are scarce, the data covered only a limited range of fire sizes, terrains, and weather conditions. This data limitation constrained the model's generalizability; performance might degrade in scenarios unlike the training set. Computational resource constraints were another limitation: the project was carried out using only moderate GPU resources and a restricted training budget, which prevented exhaustive experimentation.

Future Work

Several directions are recommended for future work to build on this project. First, collecting and using larger, more diverse datasets would strengthen the model's reliability. Collaborating with environmental agencies or research institutions to obtain real UAV footage of wildfires, or using data augmentation and simulation techniques, could provide broader training scenarios. Including infrared or thermal imagery alongside visible-light images could also be explored, as these may capture fires in challenging visibility conditions.

Second, efforts should be made to optimize the model for onboard deployment. Techniques like model pruning, weight quantization, or knowledge distillation could reduce model size and speed up inference while retaining accuracy. Testing the optimized model on actual UAV hardware (such as embedded GPUs or edge AI chips) is a crucial step. This would involve end-to-end evaluation of the system, from image capture to detection output and alert transmission. Additionally, implementing adaptive controls (for example, dynamic camera adjustments to emphasize fire signatures) could further improve detection under varying flight conditions.

Third, further testing under varied environmental conditions will be important. The model should be evaluated in low-light and night-time conditions, since wildfires can ignite outside of daylight hours. Investigating data from dawn, dusk, or heavily overcast days would test the model's robustness. Exploring temporal information, such as analysing video frames or smoke motion patterns, might help distinguish true fire activity from transient visual noise.

Finally, future work should connect the detection system to higher-level applications. Integrating the model into a real-time alert system with a user interface could demonstrate practical utility. Working with fire management teams to deploy prototype systems could provide valuable feedback and ensure that the model's capabilities align with operational needs. Such development would complete the loop from the original problem motivation to a working solution.

In summary, this project advanced the goal of fast, accurate wildfire detection in UAV imagery by enhancing YOLOv11 with targeted architectural improvements. While challenges remain, the progress made here provides a solid foundation. By linking back to the original problem statement and motivation, this work contributes to the broader effort of using technology to mitigate wildfire risk. With additional data, optimization, and field testing, the approach developed here has strong potential to aid early wildfire warning efforts.

Number of words until this point, excluding front matter: 1110.

Chapter 8

1 Bibliography

Akhloufi, M. A., Couturier, A. & Castro, N. A. C. N. A., 2021. *Unmanned Aerial Vehicles for Wildland Fires: Sensing, Perception, Cooperation and Assistance,* s.l.: MDPI.

Arafat Islam, M. I. H., 2023. *Fire Detection From Image and Video Using YOLOv5,* s.l.: arvix.

B. Ugur T ˇ oreyin, Y. D. g. a. A. E. C., 2005. *WAVELET BASED REAL-TIME SMOKE DETECTION IN VIDEO,* Ankara: s.n.

Byoungjun Kim, J. L., 2019. *A Video-Based Fire Detection Using Deep Learning Models,* s.l.: MDPI.

Cao, Y. et al., 2019. *GCNet: Non-Local Networks Meet Squeeze-Excitation Networks and Beyond,* s.l.: IEEE Xplore.

Chen, Y. et al., 2020. *Dynamic Convolution: Attention Over Convolution Kernels,* s.l.: IEEE Xplore.

Dai, X. et al., 2022. *Revisiting Sparse Convolutional Model for Visual Recognition ,* s.l.: arxiv.

Dimitropoulos, K., Barmpoutis, P. & Grammalidis, N., 2015. *Spatio-Temporal Flame Modeling and Dynamic Texture Analysis for Automatic Video-Based Fire Detection,* s.l.: IEEE Xplore.

Frizzi, S. et al., 2016. *Convolutional neural network for video fire and smoke detection,* s.l.: IEEE Xplore.

Gallagher, J., 2024. [Online]
Available at: https://blog.roboflow.com/what-is-yolo11/

Gao Xu, Y. Z. ,. Q. Z. ,. G. L. ,. Z. W. ,. Y. J. ,. J. W., 2019. *Video smoke detection based on deep saliency network,* s.l.: ScienceDirect.

Ghiasi, G. et al., 2021. *Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation,* s.l.: IEEE Xplore.

Gonçalves, L. A. O., Akhloufi, M. A. & Ghali, R., 2024. *YOLO-Based Models for Smoke and Wildfire Detection in Ground and Aerial Images,* s.l.: MDPI.

Hu, J. et al., 2017. *Squeeze-and-Excitation Networks,* s.l.: arxiv.

Ismail, et al., 2025. *A Systematic Literature Review of Vision-Based Fire Detection, Prediction and Forecasting.* s.l.:UKM.

Jadon, A. et al., 2019. *FireNet: A Specialized Lightweight Fire & Smoke Detection Model for Real-Time IoT Applications,* s.l.: arxiv.

Johnston, J. M. et al., n.d. *Satellite Detection Limitations of Sub-Canopy Smouldering Wildfires in the North American Boreal Forest,* s.l.: MDPI.

Junhui Li, R. X. a. L., 2023. *An Improved Forest Fire and Smoke Detection Model Based on YOLOv5 ,* s.l.: MDPI.
Khanam, R. & Hussain, M., 2024. *YOLOv11: An Overview of the Key Architectural Enhancements,* s.l.: arxiv.
Lee, D. H. & Chung, S.-Y., 2021. *Unsupervised Embedding Adaptation via Early-Stage Feature Reconstruction for Few-Shot Classification,* s.l.: arxiv.
Leon Augusto Okida Gonçalves, R. G. M. A. A., 2024. *YOLO-Based Models for Smoke and Wildfire Detection in Ground and Aerial Images,* s.l.: MDPI.
Liang, T. et al., 2021. *CBNet: A Composite Backbone Network Architecture for Object Detection,* s.l.: arxiv.
Li, X., Wang, W., Hu, X. & Yang, J., 2019. *Selective Kernel Networks,* s.l.: arxiv.
Li, Y., Chen, Y., Wang, N. & Zhang, Z.-X., 2019. *Scale-Aware Trident Networks for Object Detection,* s.l.: IEEE Xplore.
Ljubomir Gigović, H. R. P. ,. D. a. S. B., 2019. *Testing a New Ensemble Model Based on SVM and Random Forest in Forest Fire Susceptibility Assessment and Its Mapping in Serbia's Tara National Park,* s.l.: MDPI.
Long Sun, Y. L. T. H., 2023. *ForestFireDetector: Expanding Channel Depth for Fine-Grained Feature Learning in Forest Fire Smoke Detection,* s.l.: MDPI.
Medium, 2018. *Understanding Confusion Matrix.* [Online]
Available at: https://medium.com/data-science/understanding-confusion-matrix-a9ad42dcfd62
Misra, D., 2024. *Attention Mechanisms in Computer Vision: CBAM.* [Online]
Available at: https://www.digitalocean.com/community/tutorials/attention-mechanisms-in-computer-vision-cbam
[Accessed 2 May 2025].
Mukhriddin Mukhiddinov, A. B. A. J. C., 2022. *A Wildfire Smoke Detection System Using Unmanned Aerial Vehicle Images Based on the Optimized YOLOv5,* s.l.: MDPI.
NASA, n.d. [Online]
Available at: https://science.nasa.gov/wildfires-and-climate-change/#:~:text=%2A%20,has%20more%20than%20doubled%20worldwide
Nurmaini, D. S. & Pratama, D. A. R., 2023. *IMPROVING OBJECT DETECTION EFFICIENCY: YOLO V4 AND BACKBONE SELECTION ,* s.l.: Zendo .
Pu Li, W. Z., 2020. *Image fire detection algorithms based on convolutional neural networks,* s.l.: ScienceDirect.
Qi-xing Zhang, G.-h. L. ,. Y.-m. Z. ,. G. X. ,. J.-j. W., 2018. *Wildland Forest Fire Smoke Detection Based on Faster R-CNN using Synthetic Smoke Images,* s.l.: ScienceDirect.
Ryu, J. & Kwak, D., 2022. *A Study on a Complex Flame and Smoke Detection Method Using Computer Vision Detection and Convolutional Neural Network,* s.l.: MDPI.

Shang-Hua Gao, T. et al., 2021. *Res2Net: A New Multi-Scale Backbone Architecture,* s.l.: IEEE Xplore.
Soon-Young Kim, A. M., 2023. *Forest Fire Smoke Detection Based on Deep Learning Approaches and Unmanned Aerial Vehicle Images,* s.l.: MDPI.
Sousa, M. J., Moutinho, A. & Almeida, M., 2019. *Wildfire detection using transfer learning on augmented datasets,* s.l.: ResearchGate.
Tan, M. & Le, Q. V., 2019. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,* s.l.: arxiv.
Tan, M., Pang, R. & Le, Q. V., 2019. *EfficientDet vs. DAMO-YOLO: A Technical Comparison for Object Detection.* [Online]
Available at: https://docs.ultralytics.com/compare/efficientdet-vs-damo-yolo/
Tan, M., Pang, R. & Le, Q. V., 2020. *EfficientDet: Scalable and Efficient Object Detection,* s.l.: IEEE Xplore.
Tao, C., Zhang, J. & Wang, P., 2016. *Smoke Detection Based on Deep Convolutional Neural Networks,* s.l.: IEEE Xplore.
Ultralytics, 2023. *Explore Ultralytics YOLOv8.* [Online]
Available at: https://docs.ultralytics.com/models/yolov8/
Ultralytics, 2023. *YOLOv10: Real-Time End-to-End Object Detection.* [Online]
Available at: https://docs.ultralytics.com/models/yolov10/
Ultralytics, 2024. *Ultralytics YOLOv5.* [Online]
Available at: https://docs.ultralytics.com/models/yolov5/
Ultralytics, 2025. *Ultralytics YOLO11.* [Online]
Available at: https://docs.ultralytics.com/models/yolo11/
[Accessed 2 May 2025].
Ultralytics, n.d. *Transfer Learning with Frozen Layers in YOLOv5.* [Online]
Available at:
https://docs.ultralytics.com/yolov5/tutorials/transfer_learning_with_frozen_layers/
Vasconcelos, C., Birodkar, V. & Dumoulin, V., 2022. *Proper Reuse of Image Classification Features Improves Object Detection,* s.l.: arxiv.
Venâncio, P. & Gaia, 2022. *D-Fire: an image dataset for fire and smoke detection.* [Online]
Available at: https://github.com/gaiasd/DFireDataset
Wadii Boulila, E. A. A. A. A. K. N. M., 2024. *An Effective Weight Initialization Method for Deep Learning: Application to Satellite Image Classification,* s.l.: arxiv.
Wang, F. et al., 2017. *Residual Attention Network for Image Classification,* s.l.: IEEE Xplore.
Wang, Q. et al., 2020. *ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks,* s.l.: IEEE Xplore.
Wei, Z. et al., 2017. *Learning Adaptive Receptive Fields for Deep Image Parsing Network,* s.l.: IEEE Xplore.
Wenguang Tao 1, X. W., Yan, T., Liu, Z. & Wan, S., 2024. *ESF-YOLO: an accurate and universal object detector based on neural networks,* s.l.: NIH.

Woo, S., Park, J., Lee, J.-Y. & Kweon, I. S., 2018. *CBAM: Convolutional Block Attention Module,* s.l.: arxiv.
Wu, S. et al., 2025. *On the Study of Joint YOLOv5-DeepSort Detection and Tracking Algorithm for Rhynchophorus ferrugineus,* s.l.: MDPI.
Xin Chen, Y. X. Q. H. ,. F. Y. Z. Y. Z., 2023. *RepVGG-YOLOv7: A Modified YOLOv7 for Fire Smoke Detection,* s.l.: MDPI.
Xu, X. et al., 2023. *TranSDet: Toward Effective Transfer Learning for Small-Object Detection,* s.l.: MDPI.
Xu, Y. et al., 2024. *CNTCB-YOLOv7: An Effective Forest Fire Detection Model Based on ConvNeXtV2 and CBAM,* s.l.: MDPI.
Yin, Z. et al., 2017. *A Deep Normalization and Convolutional Neural Network for Image Smoke Detection,* s.l.: IEEE Xplore.
Yoon-Ho Kim, A. K. H.-Y. J., 2014. *RGB Color Model Based the Fire Detection Algorithm in Video Sequences on Wireless Sensor Network.* [Online].
Yue Cao, J. X. S. L. F. W. H. H., 2019. *GCNet: Non-local Networks Meet Squeeze-Excitation Networks and Beyond,* s.l.: arxiv.
Yusuf Hakan Habiboğlu, O. G. &. A. E. Ç., 2011. *Covariance matrix-based fire and flame detection method in video,* s.l.: Springer Nature.
Zhang, G., Wang, M. & Liu, K., 2021. *Deep neural networks for global wildfire susceptibility modelling,* s.l.: ScienceDirect.
Zhang, H. et al., 2020. *ResNeSt: Split-Attention Networks,* s.l.: arxiv.
Zhang, S., Li, A., Ren, J. & Li, X., 2024. *An enhanced YOLOv8n object detector for synthetic diamond quality evaluation,* s.l.: scientifc reports.

Appendix A

Design Diagrams

Layer	From	Repeats	Module	Args	Description
0	-1	1	Conv	[64, 3, 2]	Conv layer, 64 filters, stride 2 (P1/2)
1	-1	1	Conv	[128, 3, 2]	Conv layer, 128 filters, stride 2 (P2/4)
2	-1	2	C3k2	[256, False, 0.25]	C3 block, 256 channels, shortcut=False
3	-1	1	Conv	[256, 3, 2]	Conv layer, stride 2 (P3/8)
4	-1	2	C3k2	[512, False, 0.25]	C3 block, 512 channels, shortcut=False
5	-1	1	Conv	[512, 3, 2]	Conv layer, stride 2 (P4/16)
6	-1	2	C3k2	[512, True]	C3 block, 512 channels, shortcut=True
7	-1	1	Conv	[1024, 3, 2]	Conv layer, stride 2 (P5/32)
8	-1	2	C3k2	[1024, True]	C3 block, 1024 channels, shortcut=True
9	-1	1	SPPF	[1024, 5]	Spatial Pyramid Pooling Fast
10	-1	2	C2PSA	[1024]	Cross-stage PSA attention module
11	-1	1	nn.Upsample	[None, 2, "nearest"]	Upsample x2 (nearest)
12	[-1, 6]	1	Concat	[1]	Concat with layer 6 (P4)
13	-1	2	C3k2	[512, False]	C3 block, 512 channels, shortcut=False
14	-1	1	nn.Upsample	[None, 2, "nearest"]	Upsample x2 (nearest)
15	[-1, 4]	1	Concat	[1]	Concat with layer 4 (P3)
16	-1	2	C3k2	[256, False]	C3 block, 256 channels, shortcut=False (P3/8-small)
17	-1	1	Conv	[256, 3, 2]	Downsample conv
18	[-1, 13]	1	Concat	[1]	Concat with layer 13 (P4)
19	-1	2	C3k2	[512, False]	C3 block, 512 channels (P4/16-medium)
20	-1	1	Conv	[512, 3, 2]	Downsample conv
21	[-1, 10]	1	Concat	[1]	Concat with layer 10 (P5)
22	-1	2	C3k2	[1024, True]	C3 block, 1024 channels (P5/32-large)
23	[16, 19, 22]	1	Detect	[nc]	Detection heads at P3/8, P4/16, P5/32

Table 9: YOLOv11 Architectural Layer Breakdown

Model	Input Size	Parameter	Accuracy (%)
EfficientNetB0	224 × 224	4,057,253	97.95
EfficientNetB1	240 × 240	6,582,914	97.54
EfficientNetB2	260 × 260	7,777,012	96.04
EfficientNetB3	300 × 300	10,792,746	95.90
EfficientNetB4	380 × 380	17,684,570	94.13
EfficientNetB5	456 × 456	28,525,810	95.22
EfficientNetB6	528 × 528	40,973,969	98.22
EfficientNetB7	600 × 600	64,113,049	98.36

Table 10: EfficientNet Version List

Model	size (pixels)	mAP^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Table 11: YOLOv11 version List

Appendix B

Raw results output

model	precision	recall	f1	mAP50	inference_fps
CBAM_all	0.707	0.648	0.676	0.697	56.71
CBAM_backbone	0.71	0.667	0.688	0.715	57.24
CBAM_neck	0.742	0.684	0.712	0.723	57.25
CBAM_wildfire	0.766	0.693	0.728	0.75	57.31
CBAM_minimal	**0.771**	**0.698**	**0.733**	**0.752**	**57.87**
t_CBAM_minimal_100	0.747	0.675	0.709	0.731	57.74
t_CBAM_mixed_100	0.714	0.657	0.685	0.712	56.39
t_CBAM_mixed_50	0.722	0.64	0.679	0.705	57.19
t_CBAM_mixed_125	0.718	0.639	0.676	0.704	56.97
YOLOv11	0.749	0.687	0.717	0.728	56.86
ConvNext	**0.771**	0.66	0.71119	0.72	56.81
EfficientNet	0.74	0.697	0.71786	0.7	56.94

Table 12: Testing on Validation Dataset

Appendix C

Code

```python
class ChannelAttention(nn.Module):
    def __init__(self, channels, reduction_ratio=16):
        super(ChannelAttention, self).__init__()

        # Ensure reduction doesn't create channels < 8
        self.reduction_ratio = min(reduction_ratio, channels // 8)
        self.reduction_ratio = max(2, self.reduction_ratio)

        # Shared MLP for channel attention
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)

        # Shared fully connected layers
        self.mlp = nn.Sequential(
            nn.Conv2d(channels, channels // self.reduction_ratio, 1, bias=False),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels // self.reduction_ratio, channels, 1, bias=False)
        )

        self.sigmoid = nn.Sigmoid()

        # Initialize with small weights
        self._initialize_weights(0.1)

    def _initialize_weights(self, scale=0.1):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
                m.weight.data.mul_(scale)

    def forward(self, x):
        # Global average pooling
        avg_out = self.mlp(self.avg_pool(x))
        # Global max pooling
        max_out = self.mlp(self.max_pool(x))

        out = avg_out + max_out
        return self.sigmoid(out)
```

Figure 16: Channel Attention Code Block

```python
class SpatialAttention(nn.Module):
    """Spatial Attention Module for CBAM"""
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()

        # Ensure kernel size is odd for proper padding
        assert kernel_size in (3, 5, 7, 9), "Kernel size must be 3, 5, 7, or 9"
        padding = (kernel_size - 1) // 2

        # Conv layer for spatial attention
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
        self.sigmoid = nn.Sigmoid()

        # Initialize with small weights
        self._initialize_weights(0.1)

    def _initialize_weights(self, scale=0.1):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
                m.weight.data.mul_(scale)

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)

        # Concatenate features along channel dimension
        out = torch.cat([avg_out, max_out], dim=1)

        # Apply convolution and sigmoid
        out = self.conv(out)
        return self.sigmoid(out)
```

Figure 17: Spatial Attention Code Block

```python
def identify_model_sections(model):
    backbone_range = range(0, 10)
    neck_range = range(10, 23)
    return {"backbone": list(backbone_range), "neck": list(neck_range)}
```

Figure 18: Identify model architecture

```python
def get_layer_position(path):
    if re.search(r'model\.[0-5]\.', path):
        return 'backbone_early'
    elif re.search(r'model\.[6-9]\.', path):
        return 'backbone_late'
    else:
        return 'neck'
```

Figure 19: YOLO Layer position

```python
def get_all_modules(module, prefix=""):
    for name, child in module.named_children():
        current_path = f"{prefix}.{name}" if prefix else name

        # Store Conv2D modules with their path and channels
        if isinstance(child, nn.Conv2d) and hasattr(child, 'out_channels'):
            # Extract layer index if present in the path
            idx = None
            match = re.search(r'model\.(\d+)', current_path)
            if match:
                idx = int(match.group(1))

            target_layers.append({
                "path": current_path,
                "module": child,
                "channels": child.out_channels,
                "index": idx
            })

        # Recursively process child modules
        get_all_modules(child, current_path)
```

Figure 20: Recursive Call for child modules

```python
model.train(
    data=data_path,
    epochs=unfreeze_at_epoch,
    imgsz=512,
    batch=16,
    name="frozen",
    project=project_dir,
    lr0=0.001,          # Lower initial learning rate
    lrf=0.01,           # Final learning rate as a fraction of initial
    cos_lr=True,        # Cosine LR scheduler
)
```

Figure 21: Stage 1-Frozen Backbone

```python
# Unfreeze all layers by resetting requires_grad to True
for param in stage2_model.parameters():
    param.requires_grad = True

print("All layers unfrozen for fine-tuning")

# Continue training with lower learning rate
stage2_model.train(
    device='cuda',
    data=data_path,
    epochs=200-unfreeze_at_epoch,
    imgsz=640,
    batch=16,
    name="unfrozen_v2",
    project=project_dir,
    lr0=0.0001,         # Much lower learning rate for fine-tuning
    lrf=0.001,          # Final learning rate
    cos_lr=True,
    # resume=True
)
```

Figure 22: Stage 2-Unfrozen Backbone